

基于风险分位规划与储能反馈的微网购电调控

摘 要

微网需要提前购买常规电力，但光伏出力和用电需求难以准确预知，临时补购又需支付五倍电价。本文考虑储能损耗、功率和容量限制，以十分钟为时段建立购电规划，并根据实际供需调整储能运行，研究不同信息条件下的购电策略。

问题一的负荷和光伏曲线已知，模型以日购电费最小为目标，要求每段供需平衡、储电量处于安全范围，且日末恢复初始库存。在允许舍弃余电的条件下，证明充放电互斥约束可以等价松弛，再用线性规划求解，并以混合整数规划复核。最优日购电费为 35126.95 元，比相同条件下的无储能方案降低 26.90%。

问题二先对过去相同星期的负荷和近期光伏数据作加权预测，再将历史误差分位数加入净负荷预测，为供电缺口预留电量。以此为输入，通过线性规划确定零时购电计划；实际运行时，余电优先充入电池，缺电时先放电，仍有缺口再紧急补购。根据 1 月的费用比较，取风险分位为 0.80，此后固定使用，日末实际剩余电量用于次日规划。2025 年 2—12 月总费用为 1390.82 万元。

问题三利用每日发布的光伏预报调整购电。先将整点预报插值为十分钟数据，与历史光伏预测加权融合；在 6 时、12 时和 18 时，结合最近的负荷观测及当前储电量，重新求解剩余时段的线性规划。模型以增购费用最小为目标，调整费按最终交付量与零时计划的差额计算。融合权重和风险分位均由 1 月费用选定，正式期总费用为 1331.58 万元，比问题二降低 4.26%。保持三次调整机会相同的对照表明，6 时和 12 时的新预报有助于降费，18 时的新预报未显示明显收益。

问题四进一步考虑电价波动。根据过去相同星期的电价形成预测；允许日内调整时，再用最近的实际价格修正预测水平。将预测价作为购电规划的费用系数，最终按交付时的实际价格结算。零时固定计划和日内调整两种方案的总费用分别为 1467.58 万元和 1404.49 万元，日内调整降费 4.30%。在所检验的预测价差扰动下，两种方案的费用排序保持稳定。

各方案均通过供需平衡、储能约束、信息使用顺序和费用核验。问题一得到确定性最优解，问题二至四得到可按当时信息执行的预测与反馈策略。经济比较基于 2025 年数据，其他年度的适用性还需进一步检验。

关键字：微网 储能调度 风险分位 因果预测 滚动评估

一、 问题重述

1.1 研究背景与任务

本文研究含光伏和储能设备的微网。日常用电由光伏、提前约定的外网购电及储能共同供应，电力不足时按对应电价的五倍紧急购电，富余电力不向外网出售。储能额定容量为 12000 kWh，允许储电量为 1200–10800 kWh，充放电最大功率均为 5000 kW。

四个问题依次增加可用信息和购电调整机会。问题一根据给定预测曲线，求日初、日末储电量相同时的最低日购电费；问题二根据历史数据制定每日零时计划，运行时随实际供需调整储能；问题三加入零时、6 时、12 时和 18 时发布的光伏预报，允许修改尚未交付的计划，并分析新预报的作用；问题四进一步考虑电价波动。各问还需按要求给出指定时段与四个指定日期的详细调度表。

1.2 数据整理与时间约定

附件一包含 144 个时点的电价及典型负荷、光伏预测，附件二和附件四包含 2025 年 365 天、每天 144 个时点的实际功率和电价，附件三包含每日 4 次、每次未来 24 小时的光伏预报。检查后，日期连续，所需数值均有效，无缺失或负数。正式评价从 2025 年 2 月 1 日至 12 月 31 日，共 334 天、48096 个十分钟区间；1 月用于积累历史数据、选择参数，并通过连续模拟确定正式期的初始储电量，以下称为热启动。

将 00:10 记录视为 00:00–00:10 区间的平均功率，单段电量即该行功率乘以 $\Delta = 1/6$ 小时。这与输入以区间结束时刻标记的方式一致。题目没有明确区分瞬时采样和平均功率，因此另用梯形积分检验这一解释对问题一的影响。原结果模板的标签整体晚十分钟，输出按原数据次序统一标为 00:00–00:10 至 23:50–24:00。

附件三第 k 个预报值对应发布时间之后第 k 小时的光伏出力。本文在相邻目标整点之间作线性插值，并以发布时刻之前最后一个已完成区间的光伏观测作为起点。6 时发布的新预报从 6:00–6:10 区间起用于制定计划，已交付的电量按此前决策记录。

附件一的典型负荷与附件二全年同刻均值几乎一致，用它预测问题二至四的负荷会间接包含未来数据。因此，后续各问的负荷和光伏预测均从已完成日期构造，附件一仅提供固定电价。问题一依题意直接采用附件一的给定预测。

二、 模型的假设

1. 每个十分钟区间内功率近似恒定，控制器能够根据当段实际负荷和光伏及时调节储能。依据题目给定条件，暂不计响应延迟、输电限制、电池老化费用及自放电。
2. 充、放电效率分别取 0.9，因而往返效率为 0.81；储能的充放电功率在交流母线侧

计量。将 90% 解释为往返效率的情形另作敏感性比较。

3. 微网不售电，外网购电量按题目条件不设功率上限。无法吸收的光伏可以不利用，已付费的计划电量也可以不接收，但费用仍照付。两类余量合称未利用电量。
4. 问题一要求日初、日末储电量相同。问题二至四从 1 月 1 日 6000 kWh 开始连续运行，每日规划将日末预测储电量至少 6000 kWh 作为恢复目标；实际储电量由供需和控制动作决定，并直接传至次日。
5. 波动电价在交付时可知，未来价格需要根据历史数据预测。规划采用预测价，费用按对应交付区间的实际价格结算，本文据此解释题目中的“交易时刻”。

三、 符号说明

表 1 主要符号与单位

符号	含义	单位
d, t	日期索引与日内十分钟区间索引, $t = 0, \dots, 143$	—
L_t, V_t	当段实际负荷电量和光伏电量	kWh
g_t, q_t	零时原始购电计划与最终常规交付量	kWh
c_t, d_t^{bat}	母线侧充电输入量与放电输出量	kWh
S_t	区间开始时的实际储电量	kWh
e_t, w_t	紧急购电与未利用电量	kWh
$p_t, \hat{p}_t$	实际结算价与决策时预测价	元/kWh
$\hat{N}_t, \tilde{N}_t$	点预测净负荷与风险调整净负荷	kWh
λ, α	附件三预测权重与风险分位参数	—

为简化以下公式，放电量记为 $b_t = d_t^{\text{bat}}$ ，从而避免与日期索引混淆。

四、 问题分析

4.1 预测、购电计划与实时执行的分工

问题一的输入曲线已知，可以直接优化全天购电和储能安排。问题二至四需要先预测再规划，实际需求偏离预测时，还要调整充放电并补购缺少的电力。本文据此安排预测、常规购电规划和实时执行三个环节。一次规划的最优解只对应当次预测，全年效果还取决于预测误差及后续运行。由于储能可以在时段间转移电量，同样的误差在不同库存和电价下会产生不同支出，故主要按实际费用评价策略，并以预测精度辅助解释。

4.2 四个问题的递进关系

问题一先建立电量平衡、储能损耗及日循环约束，再判断充放电互斥条件能否作等价线性松弛。问题二沿用这些物理关系，根据历史预测误差修正净负荷，以应对紧急购电价格较高的风险，并在运行中根据实际供需调节储能。

问题三既增加光伏预报，又允许日内修改购电，需要通过对照辨别两者各自的作用。多次修改的费用还取决于合约如何累计调整量，因而需比较最终净调整和逐笔成交两种解释。问题四进一步预测未来电价，使购电和储能安排能随预期价格变化，并按交付时的实际电价检验费用。

4.3 方法选择及相关研究

如何在不同时段分配有限储电量，是光伏储能经济调度及日前、日内协调优化的共同问题^[1,2]。模型预测控制在获得新观测后重新预测并优化未来一段时间，每次只执行第一个动作。确定性滚动控制采用一组预测，多情景鲁棒控制则考虑多种可能情况^[3]。动态电价研究还将实际可执行的预测控制，与已知未来全部数据时的表现作对照^[4]。

多时间尺度模型预测控制还可结合分布鲁棒优化，处理未来场景及其概率分布的不确定性^[5]。本文根据历史误差分位修正各段净负荷，再用线性规划制定购电计划，计算规模较小。针对这种近似可能带来的影响，分别比较历史预测、不同预报来源、交易合约及储能反馈方式。经验分位的含义和限制在问题二中具体说明。

五、模型的建立与求解

5.1 问题一：确定性购电规划

5.1.1 输入、假设与变量

附件一给出了同一天 144 个十分钟区间的负荷、光伏预测和电价，本问据此安排各段购电和充放电。分时电价下的储能经济调度已有相关研究^[1]。本题还要求日初和日末储电量相同，因此需要统筹全天，确保低价时存下的电量能够满足后续使用和日末恢复的需要。

记区间集合为 $\mathcal{T} = \{0, \dots, 143\}$ ，以 $L_t = P_t^L \Delta$ 、 $V_t = P_t^V \Delta$ 表示负荷和光伏电量。每段设置购电量 g_t 、充电量 c_t 、放电量 b_t 、未利用电量 w_t 及段末储电量 S_{t+1} 。五类变量均为连续量，单位为 kWh；电价 p_t 的单位为元/kWh。上一段结束时的储电量作为下一段的初始值。

全部输入已知时，零时即可安排满足需求的常规购电，因此本问无需紧急补购。充、放电效率各取 0.9，余电不利用时不另收费，其余物理条件采用题给参数。效率的不同解释及可用容量对费用的影响，在本问末尾检验。

5.1.2 目标函数与分条约束

以全天外网购电支出最小为目标。光伏没有题给的使用费用，储能损耗通过能量递推体现，不再额外添加一项损耗电费：

$$\min C_1 = \sum_{t \in \mathcal{T}} p_t g_t. \quad (1)$$

目标函数中电价乘电量得到以元计的支出。按十分钟时段换算电量后，依次建立以下六类约束。

a) 供需平衡。当段购电、光伏和储能放电应覆盖负荷与充电需求。引入未利用电量 w_t ，将题目中的“不低于”写成等式：

$$g_t + V_t + b_t = L_t + c_t + w_t, \quad t \in \mathcal{T}. \quad (2)$$

$w_t \geq 0$ 包括无法吸收的光伏和不接收的已付费外网电量。外网电费按计划购电量 g_t 支付，未利用部分的费用也计入目标函数。

b) 储电量递推。充电量和放电量均在交流母线侧计量，输入 1 kWh 只能增加 0.9 kWh 库存，输出 1 kWh 则需要消耗 1/0.9 kWh 库存：

$$S_{t+1} = S_t + \eta_c c_t - b_t / \eta_d, \quad \eta_c = \eta_d = 0.9. \quad (3)$$

144 个区间对应 145 个状态， S_t 和 S_{t+1} 分别为本段开始和结束时的储电量。

c) 充放电功率限制。由于决策变量是电量，5000 kW 的功率上限必须先乘十分钟的时长：

$$0 \leq c_t, b_t \leq P_{\max} \Delta, \quad P_{\max} \Delta = 5000/6 \text{ kWh}. \quad (4)$$

即使储电量充足，单段放电仍不能超过 833.33 kWh；功率和容量分别约束放电速度与可持续时间。

d) 安全库存限制。在每个区间边界均需检查

$$1200 \leq S_t \leq 10800, \quad t = 0, \dots, 144. \quad (5)$$

额定容量为 12000 kWh，扣除安全边界后，可调度的储电量范围为 9600 kWh。

e) 日循环边界。给定初始库存为 6000 kWh，问题一每天重复同一组输入，因此要求

$$S_0 = S_{144} = 6000. \quad (6)$$

这一等式要求当天消耗的初始储电量在日末得到补充，使所得计划可以逐日重复运行。

f) 购电方向及运行模式。购电和余量均非负，同一时段的充电与放电互斥：

$$g_t, w_t \geq 0, \quad c_t b_t = 0. \quad (7)$$

前两个不等式限制购电和未利用电量的取值，乘积约束保证每段至多执行充电或放电中的一种动作。

5.1.3 汇总模型与线性化

将目标和约束合并，得到问题一的规划模型：

$$\begin{aligned} (\text{P}_1) \quad & \min \sum_{t=0}^{143} p_t g_t, \\ \text{s.t.} \quad & g_t + V_t + b_t = L_t + c_t + w_t, \\ & S_{t+1} = S_t + 0.9c_t - b_t/0.9, \\ & 0 \leq c_t, b_t \leq 5000/6, \quad g_t, w_t \geq 0, \\ & 1200 \leq S_t \leq 10800, \quad S_0 = S_{144} = 6000, \\ & c_t b_t = 0 \quad (t \in \mathcal{T}). \end{aligned} \quad (8)$$

库存限制还包括日末 $t = 144$ 。模型中的非线性项来自充放电互斥条件。引入二元变量 y_t ，以 $c_t \leq (5000/6)y_t$ 、 $b_t \leq (5000/6)(1 - y_t)$ 代替乘积约束，即得到混合整数线性规划。本题允许不利用余电，下面证明在此条件下可用连续线性规划求解。

考虑松弛模型中某一同时充放电的时段，令

$$\delta = \min\{c_t, b_t/(\eta_c \eta_d)\}, \quad c'_t = c_t - \delta, \quad b'_t = b_t - \eta_c \eta_d \delta. \quad (9)$$

再令 $w'_t = w_t + (1 - \eta_c \eta_d)\delta$ 。该变换使 c'_t 、 b'_t 中至少一个为 0，并保持

$$\eta_c c'_t - b'_t / \eta_d = \eta_c c_t - b_t / \eta_d. \quad (10)$$

变换前后的储电量相同，充放电功率均未增加，增加的未利用量恰好补足能量平衡的差额。购电量 g_t 也相同，故费用不变。这样，线性松弛的任一最优解都能转成满足互斥条件、费用相同的解，两种模型的最低电费一致。

上述等价关系以允许不利用余电、额外充放电没有奖励为前提。加入弃电惩罚、禁止弃电或循环补贴后，需要重新判断变换是否保持可行和等费用。

5.1.4 求解步骤与最优性核验

去掉互斥乘积后，使用 $\mathbf{x} = (\mathbf{g}, \mathbf{c}, \mathbf{b}, \mathbf{S}_{1:144}, \mathbf{w})$ 作为求解向量，共 720 个连续变量。每个时段有一条平衡等式和一条状态等式，共 288 条；容量、功率和日末等式通过变量

边界给定。系数矩阵只连接本段动作与相邻库存，适合使用稀疏矩阵存储。

计算通过 SciPy 的 HiGHS 接口完成^[6]；所用求解器包含对偶修正单纯形等线性优化方法^[7]。求解过程为：

1. 将 144 行功率乘以 1/6，形成负荷、光伏电量与目标系数向量，检查单位和时段顺序。
2. 按两类等式逐行装配系数矩阵，设置充放电、库存及首末库存边界。
3. 求解纯电费 LP，得到最低费用；另以微小选解项 $10^{-6} \sum_t (c_t + b_t) + 10^{-7} \sum_t w_t$ 选定调度方案。
4. 将选解后的实际电费与纯电费下界比较，不把选解项计入购电账单。
5. 独立建立含 144 个模式变量的 MILP，检查目标差、整数性和物理残差。
6. 将通过核验的十分钟结果按指定区间聚合，得到购电与储能的汇总结果。

纯电费 LP 与独立 MILP 的结果均为 35126.948589 元，MILP 最优性间隙为 0，最大约束残差约 7.62×10^{-12} kWh。选定调度也达到该费用。目标下界、实际调度与整数模型的结果一致，确认了本问在给定输入下的最优性。

5.1.5 购电结果与储能作用

最优日购电量为 59,482.70 kWh，全天费用为 35,126.95 元。表2按题面表 1 给出六个十分钟区间和全天购电结果；表3按题面表 2 列出六个四小时区间的充放电量以及首末库存。两表电量单位均为 kWh，购电费单位为元；充放电区间接题面顺序从左至右、从上至下排列。

表 2 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	480.41	14:00–14:10	0.00
16:00–16:10	445.43	18:00–18:10	531.89	20:00–20:10	0.00
全天购电量		59,482.70	全天购电费		35,126.95

表 3 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4,500.00	0.00	4:00–8:00	833.33	6,365.84
8:00–12:00	4,787.96	1,703.00	12:00–16:00	5,286.04	91.10
16:00–20:00	0.00	5,780.13	20:00–24:00	5,333.33	2,859.87
0:00 储电量		6,000.00	24:00 储电量		6,000.00

为解释购电时段的安排，图1同时给出购电、充放电、库存和电价。库存上限使低价区间不能无限购电，功率上限使高价区间不能一次性释放全部库存，日循环条件又要求当天用掉的库存得到补充。

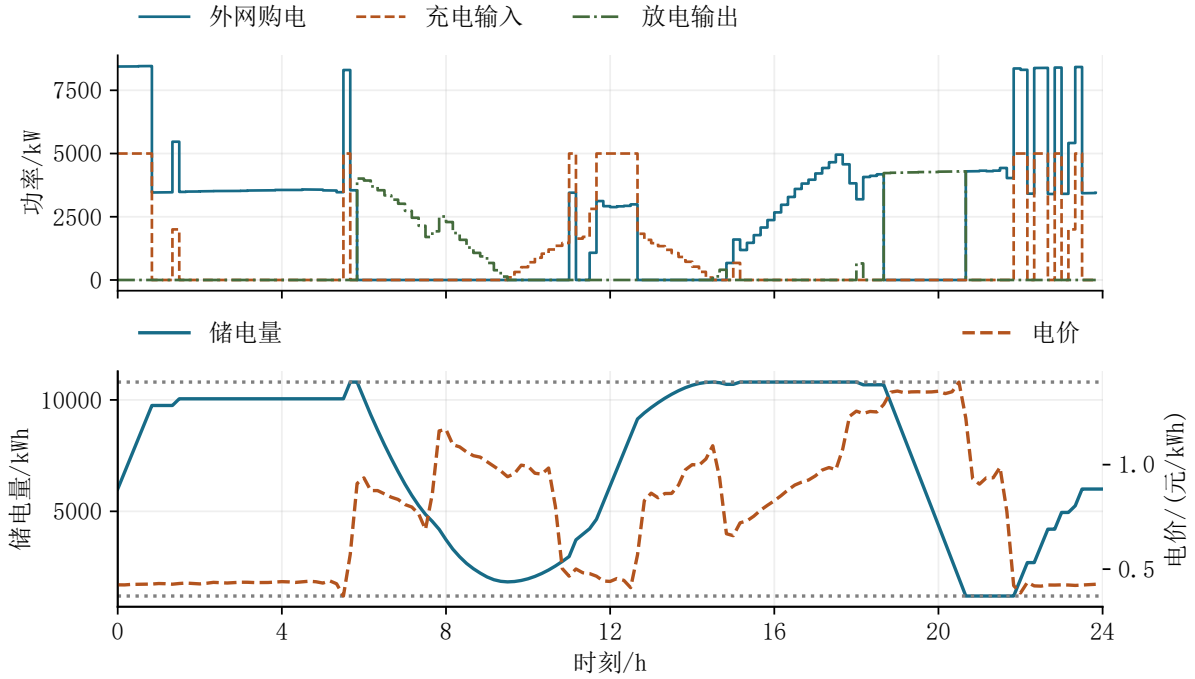


图1 问题一的购电、储能及电价关系

在相同已知曲线下，关闭储能并逐段购买正净负荷，费用为 48,052.05 元。储能使当日电费减少 12,925.10 元，降幅 26.90%。这是同一确定输入和同一费用规则下的比较，节省主要来自光伏余量利用与跨时段购电转移。

全天充电输入为 20,740.67 kWh，放电输出为 16,799.94 kWh，两者相差 3,940.73 kWh。由日循环边界对式(3)求和可得

$$\sum_t b_t = \eta_c \eta_d \sum_t c_t = 0.81 \sum_t c_t. \quad (11)$$

充放电电量的差额来自往返损耗。若从低价段向高价段转移 1 kWh 电量，高价需超过低价除以 0.81，转移后的购电费用才会更低；实际安排还受光伏出力、库存和功率限制。四小时汇总中充电和放电同时为正，表示该区间内的不同十分钟段分别发生了充电和放电。

5.1.6 容量、功率与效率灵敏度

固定附件一曲线和 6000 kWh 首末库存，分别改变可用库存幅度 B_u 、充放电功率上限 $P_{\max}$ 和单向效率 η ，每次只改变一个参数并重新优化。容量实验令库存范围为 $[6000 - B_u/2, 6000 + B_u/2]$ ，通过扩大或缩小可用范围比较容量的作用。各组初始电量相同，费用变化均以主方案为基准，明细见附录B.1表55。可用容量增加 20% 时，费用

下降 3.58%；同幅度提高功率仅降费 0.12%，说明当前配置下容量的限制更明显。单向效率由 0.90 提高到 0.95 后，充放电损耗减少，费用下降 3.87%。各组均通过物理约束和费用核验，LP 目标与对偶下界一致。这些结果反映设备参数对购电支出的影响，扩容回收期还需结合投资成本计算。

若 90% 指往返效率，两侧取 $\sqrt{0.9}$ ，费用为 33,801.50 元；采用梯形积分时为 35,042.54 元。二者仅作参考对照，主结果仍采用区间均值和双侧 0.9 效率。

5.2 问题二：固定计划与实时反馈

5.2.1 信息条件与决策变量

本问电价已知，当天负荷和光伏则需要预测。零时确定的购电量全部付费，未使用的部分不退款；供电不足时又需按五倍电价补购。因此，计划电量过多会增加浪费，过少则可能带来高额紧急费用。

记 $\mathcal{H}_d$ 为第 d 天零时已有的历史观测， $\hat{N}_{d,t}$ 为净负荷点预测。规划变量包括固定购电计划 $g_{d,t}$ ，以及预测的充放电量 $\hat{c}_{d,t}$ 、 $\hat{b}_{d,t}$ 、库存 $\hat{S}_{d,t}$ 和余量 $\hat{w}_{d,t}$ 。带帽变量描述预测条件下的运行安排，实际充放电、库存、紧急购电和余量分别用 c, b, S, e, w 记录。

本问沿用问题一的效率、母线侧计量、容量和功率约束。实际运行从 1 月 1 日 6000 kWh 开始，储电量逐日连续传递。为给次日留出一定储备，规划中设置日末恢复目标 $B = 6000$ kWh，即预测日末库存至少 6000 kWh；实际日末库存由当天供需和反馈动作决定。

5.2.2 目标函数与风险净负荷

以正式期实际购电支出评价策略：

$$C_2 = \sum_{d=31}^{364} \sum_{t=0}^{143} (p_t g_{d,t} + 5p_t e_{d,t}). \quad (12)$$

日期索引从 1 月 1 日 $d = 0$ 开始。零时尚不清楚紧急购电量 $e_{d,t}$ ，本文用历史预测误差修正净负荷，再根据修正值安排常规购电。执行时按实际需求调节储能并补购缺电，以此近似处理预测误差带来的风险。

负荷点预测考虑每周用电规律。历史数据满一周后，取此前最多四个与目标日星期相同的日期，对其同刻负荷加权：

$$\hat{P}_{d,t}^L = \frac{\sum_{k=1}^{K_d} 0.55^{k-1} P_{d-7k,t}^L}{\sum_{k=1}^{K_d} 0.55^{k-1}}, \quad K_d = \min\{4, \lfloor d/7 \rfloor\}. \quad (13)$$

历史不足 7 天时改用此前最多 7 天，以 0.8^{k-1} 衰减加权。光伏采用较短的历史窗口：

$$\hat{P}_{d,t}^{V,H} = \frac{\sum_{k=1}^{\min(7,d)} 0.75^{k-1} P_{d-k,t}^V}{\sum_{k=1}^{\min(7,d)} 0.75^{k-1}}, \quad d \geq 1. \quad (14)$$

首日没有历史数据，负荷取 4500 kW、光伏取 0，用于初始化。两类功率预测相减后乘 Δ ，得到 $\hat{N}_{d,t}$ 。以后每天均按已完成日期更新预测，从而满足零时只能使用过去观测的要求。

对每个已完成日期 j ，保存当时预测与实际净负荷的残差 $R_{j,t} = N_{j,t} - \hat{N}_{j,t}$ 。在 $\max(7, d-28) \leq j < d$ 内，将同刻及前后各 3 段残差合并，记为 $\mathcal{R}_{d,t}$ ；区间端点以最近端点复制补齐。规划输入为

$$\tilde{N}_{d,t} = \hat{N}_{d,t} + Q_\alpha(\mathcal{R}_{d,t}). \quad (15)$$

分位数在排序后的样本间线性插值，没有可用残差时取修正量为零。较大的 α 通常增加提前备电，也可能留下更多已付费却未利用的电量。这里 α 表示历史单段误差的分位位置，系统的供电可靠性还取决于误差是否连续发生以及当时可用的储电量。

由此，零时实际求解的目标为

$$\min \hat{C}_{2,d} = \sum_{t=0}^{143} p_t g_{d,t}. \quad (16)$$

该目标用于选择修正净负荷下的购电计划，式(12)则计算运行后真正支付的费用。选取分位参数时，按后者比较历史表现，使提前备电和紧急补购的代价都得到考虑。

5.2.3 约束条件与汇总模型

a) 预测供需平衡。以 $\tilde{N}_{d,t}$ 替代未知的实际净负荷，要求

$$g_{d,t} + \hat{b}_{d,t} = \tilde{N}_{d,t} + \hat{c}_{d,t} + \hat{w}_{d,t}. \quad (17)$$

预测中的供电需求由常规购电 g 和储能共同满足。交付时若仍有实际缺口，再按五倍电价紧急补购。

b) 储能递推与边界。预测态使用式(3)的同效率关系，并满足 $0 \leq \hat{c}, \hat{b} \leq 5000/6$ 、 $1200 \leq \hat{S} \leq 10800$ 。允许未利用余量，故可采用问题一证明的线性松弛。

c) 起止库存。以当日实际储电量为规划起点，并设置日末恢复目标：

$$\hat{S}_{d,0} = S_{d,0}, \quad \hat{S}_{d,144} \geq B, \quad B = 6000. \quad (18)$$

保留日末库存，是为了减少当日规划用尽储能对次日的影响。两阶段调度研究也考虑日前库存边界与日内决策的衔接^[8]。本文将 6000 kWh 设为恢复目标，其影响在后续库存

敏感性比较中检验。

d) 固定计划和可用信息。全天 $q_{d,t} = g_{d,t}$ ，零时确定后固定执行。计划 g_d 由历史观测 $\mathcal{H}_d$ 及实际日初库存决定，当天新获得的观测用于储能反馈和紧急补购。

将上述关系合并，省略日期下标后得到

$$\begin{aligned}
 (\mathbf{P}_2) \quad & \min \sum_{t=0}^{143} p_t g_t, \\
 \text{s.t.} \quad & g_t + \hat{b}_t = \tilde{N}_t + \hat{c}_t + \hat{w}_t, \\
 & \hat{S}_{t+1} = \hat{S}_t + 0.9\hat{c}_t - \hat{b}_t/0.9, \\
 & 0 \leq \hat{c}_t, \hat{b}_t \leq 5000/6, \quad g_t, \hat{w}_t \geq 0, \\
 & 1200 \leq \hat{S}_t \leq 10800, \\
 & \hat{S}_0 = S_{d,0}, \quad \hat{S}_{144} \geq 6000.
 \end{aligned} \tag{19}$$

区间约束作用于 $t = 0, \dots, 143$ ，库存边界还包括日末。计划确定后，每段按实际充放电更新储电量，次日继续使用实际日末库存。

5.2.4 参数选择与执行算法

候选分位为 $\{0.50, 0.65, 0.70, 0.80, 0.90, 0.95\}$ 。各候选从 1 月 1 日 6000 kWh 开始独立模拟，按 1 月 15—31 日的实际费用比较，选得 $\alpha = 0.80$ 。正式运行所用的 1 月热启动预先设为 0.80，2 月起继续使用该参数。此后仅随日期更新历史数据，参数保持固定。

当段实际负荷和光伏到达后，定义常规电力相对净需求的余量 $z_t = g_t - (L_t - V_t)$ 。若 $z_t \geq 0$ ，尽可能将余量存入电池：

$$\begin{aligned}
 c_t &= \min\{z_t, 5000/6, (10800 - S_t)/0.9\}, \\
 b_t = e_t &= 0, \quad w_t = z_t - c_t.
 \end{aligned} \tag{20}$$

若 $z_t < 0$ ，先在功率与库存允许范围内放电，其余缺口紧急购入：

$$\begin{aligned}
 b_t &= \min\{-z_t, 5000/6, 0.9(S_t - 1200)\}, \\
 c_t = w_t &= 0, \quad e_t = -z_t - b_t.
 \end{aligned} \tag{21}$$

充电量分别由供需余缺、功率上限和可用库存中的最紧限制决定。按式(3)更新储电量后，即可满足平衡和安全边界。按题目所给的紧急购电条件，放电后的剩余缺口均能补足。

每天按以下步骤运行：

1. 在零时读取前一日末库存和已完成日期观测，计算当日点预测。
2. 按固定 α 提取历史残差分位，构造 144 段风险净负荷。

3. 求解 (P_2), 确定全天 g_t , 记录预测条件下的储能安排。
4. 在每个交付区间接收实际净负荷, 按式(20)或(21)执行, 并更新真实库存。
5. 对原计划和实际紧急电分别计费, 检查当段供需平衡; 日末库存直接传入次日。
6. 累计 2—12 月费用, 并汇总指定日期的购电计划、储能运行与紧急购电事件。

这一反馈按当前余缺决定充放电。后文再加入对未来电价的考虑, 比较储能动作变化后的费用。

5.2.5 全年与指定日期结果分析

正式期 334 天总费用为 13,908,175.95 元, 其中紧急电费 616,550.23 元, 约占总费的 4.43%。表4给出费用和电量分解, 表5汇总四个指定日期。正文按题面表 1、表 2 展示 9 月 23 日, 其他日期及四日期紧急事件、账单见附录A.1, 与 result2.xlsx 对应。

表 4 问题二的累计结果 (2025 年 2—12 月)

指标	数值
原计划费/元	13,291,625.72
紧急费/元	616,550.23
总费用/元	13,908,175.95
常规购电/kWh	21,712,548.31
紧急购电/kWh	102,787.48

表 5 问题二的指定日期结果 (2025 年)

日期	常规购电/kWh	紧急购电/kWh	全天总费用/元
03-20	68,386.72	0.00	41,537.79
06-21	37,707.19	0.00	22,414.49
09-23	68,094.10	892.40	47,814.59
12-21	99,788.74	0.00	64,446.61

各日期购电表中的购电量为零时确定的常规计划, “全天购电费”为该常规计划费用; 紧急购电另列, 含紧急费的总支出见全天账单。电量单位均为 kWh, 费用单位为元。储能表按六个四小时区间求和, 并列当日 0:00 与 24:00 的真实库存。表内保留两位小数, 计算使用完整精度, 逐项舍入后可能产生 0.01 的加总差。

附录紧急购电表将大于 10^{-7} kWh 的连续十分钟段合并为事件, 终点为最后一段结束时刻; “无、0.00”表示当天没有紧急购电, 后续空白表示没有更多事件。24:00 与 0:00+1 含义相同。后续各问沿用上述口径。2025.9.23 的购电和储能结果分别见表6和表7。

表 6 微网在指定时间段的购电量及全天的购电量和购电费（2025.9.23）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	499.87	14:00–14:10	0.00
16:00–16:10	531.67	18:00–18:10	775.50	20:00–20:10	0.00
全天购电量		68,094.10	全天购电费		42,079.63

表 7 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.9.23）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4,635.38	107.23	4:00–8:00	1,019.68	6,091.91
8:00–12:00	5,459.91	1,896.62	12:00–16:00	4,468.06	602.55
16:00–20:00	79.59	5,891.78	20:00–24:00	5,549.16	2,391.13
0:00 储电量		5,971.67	24:00 储电量		6,194.24

四个指定日期中，9 月 23 日出现 892.40 kWh 紧急购电，其余三个日期没有。该日总费 47814.59 元，其中 5734.95 元来自紧急购电。图2选取这一日期展示风险计划、实际净需求及库存的关系，便于检查缺口发生的位置。

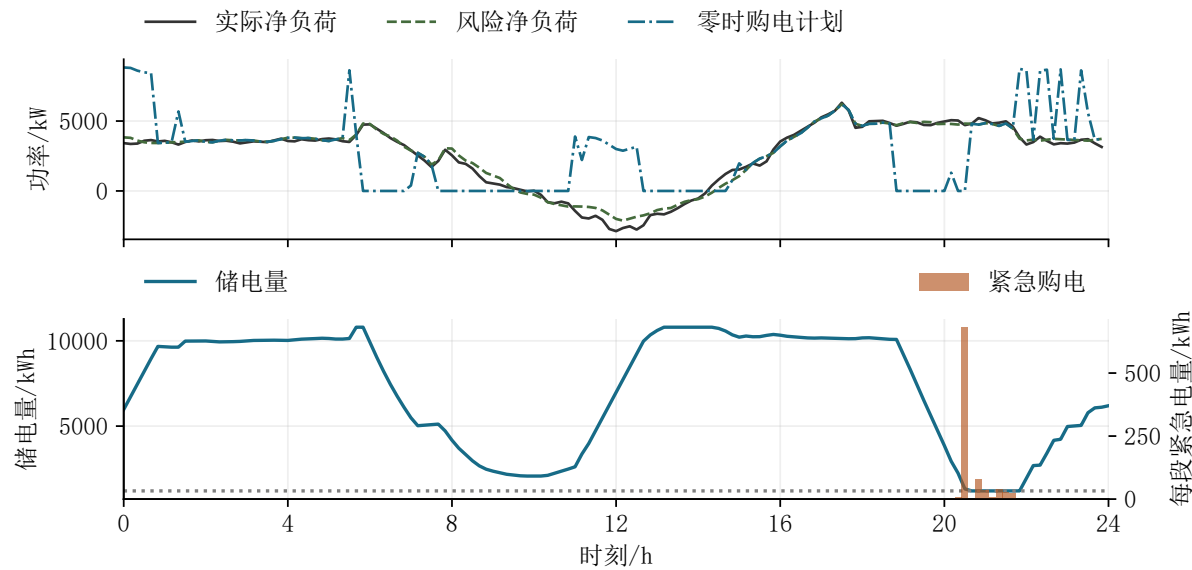


图 2 问题二在 9 月 23 日的固定计划与实际执行

预测低估净负荷时，电池可承担部分缺口；一旦库存或功率达到放电限制，便需紧急购电。因此，预测误差的费用影响取决于缺口出现时的电价和剩余放电能力。

全年未利用电量为 2,646,069.43 kWh，其中包括已付费的常规电和未吸收的光伏。全年 115 天发生紧急购电；期初、期末库存分别为 6,840.33 kWh 和 6,258.37 kWh。费用按实际支出统计，库存价值未折算进账单。

5.2.6 风险参数灵敏度检验

采用相同预测方法、反馈规则及1月热启动，将正式期分位改为0.50、0.70和0.90，与主参数0.80比较。表8同时列出总费、紧急电量及年末库存，以观察增加备电的收益和代价。

表8 问题二的分位参数扰动（2—12月）

α	总费用/元	紧急购电/kWh	年末储电量/kWh
0.5	15,393,484.98	526,317.07	5,890.36
0.7	14,015,177.03	190,439.93	6,060.07
0.8	13,908,175.95	102,787.48	6,258.37
0.9	14,140,389.51	40,231.09	6,507.64

分位从0.50提高至0.80时，提前备电减少了高价补购，总费下降。继续提高至0.90，紧急电量进一步减少，总费却增加约23.22万元，新增常规电费已超过省下的补购费用。主参数在所列候选中费用较低；由于各方案年末库存不同，表中比较的是各自连续运行的实际支出，尚未求得连续参数范围内的最优值。增加日内调整机会后，还需重新按历史费用选参。

5.3 问题三：日内调整购电规划

5.3.1 新增信息与交易变量

问题三在固定电价下，每天00、06、12、18点可获得未来整点的光伏预报，并可在后三个时刻修订尚未交付的购电量。新预报有助于更新对光伏出力的判断，日内调整则可以结合实际库存和近期负荷重新安排购电。为判断两者各自的作用，后文在相同调整时刻下比较不同预报的使用效果。

记零时计划为 $g_{d,t}$ ，发布时间对应的区间索引为 $s \in \{0, 36, 72, 108\}$ ，在 s 修订的剩余计划为 $q_{d,t}^{(s)}$ ，最终交付的常规电量为 $q_{d,t}$ 。规划根据预测安排购电，运行时仍按问题二的规则调节储能。可用信息 $\mathcal{H}_{d,s}$ 包括此前日期的历史、当日 s 之前已完成区间的观测，以及截至 s 已发布的预报。

本问沿用余电可以不利用、原计划款不返还的假设。主要计算按每段最终交付量与零时计划的差额收费，允许撤回尚未交付的追加申报。下文将这一解释称为主合约，另比较追加电量成交后不可撤回、每笔分别收费的情形。

5.3.2 费用目标与预报融合

总费用由原计划、调整和紧急购电三项组成：

$$C_3 = \sum_{d,t} [p_t g_{d,t} + 1.5p_t (q_{d,t} - g_{d,t})_+ + 0.5p_t (g_{d,t} - q_{d,t})_+ + 5p_t e_{d,t}]. \quad (22)$$

其中 $(x)_+ = \max(x, 0)$ ，求和覆盖正式期。增购 1 kWh 额外支付 $1.5p_t$ ；减购 1 kWh 时，原计划款照付，另加 $0.5p_t$ 违约费，即原价的 50%。

零时按 (P_2) 制定计划，净负荷预测采用已发布预报与历史数据的融合结果。日内 $s > 0$ 时，原计划费用已经确定，因此重新规划以剩余时段的增购费用最小为目标：

$$\min \hat{C}_{3,d,s} = \sum_{t=s}^{143} 1.5p_t (q_{d,t}^{(s)} - g_{d,t}). \quad (23)$$

本式采用 $q_{d,t}^{(s)} \geq g_{d,t}$ 的约束，理由见下节。与问题二相同，未来紧急购电的风险通过净负荷分位修正来处理，实际费用则在运行后核算。

附件三中， h 点发布的第 k 列预报对应 $h+k$ 点。每次收到预报后，以最近一个已完成区间的光伏观测为插值起点，依次连接未来整点值，得到十分钟预测 $\hat{P}^{V,F}$ ，并取其中截至当日 24 点的部分。

采用两来源线性融合：

$$\hat{P}_{d,t}^V = (1 - \lambda) \hat{P}_{d,t}^{V,H} + \lambda \hat{P}_{d,t}^{V,F}, \quad 0 \leq \lambda \leq 1. \quad (24)$$

不同来源的预测误差可能相互补充，因而可通过组合改善预测^[9]，光伏预测中也已有自适应组合的研究^[10]。本文采用上述加权形式，并按实际购电费用选择权重，使预测效果能够通过最终支出评价。

为避免近期观测使预测水平发生过大跳变，先定义区间截断函数：

$$\text{clip}(x, a, b) = \min\{b, \max\{a, x\}\}, \quad a \leq b. \quad (25)$$

其中 x 为待修正的数值， a 、 b 分别为下限和上限：小于 a 时返回 a ，大于 b 时返回 b ，在区间内则保留 x 。本文将它作用于无量纲修正比例，例如 $\text{clip}(1.40, 0.75, 1.25) = 1.25$ 。

在日内发布时刻 $s \in \{36, 72, 108\}$ ，再用此前 18 个已完成区间，即最近 3 小时的观测修正负荷水平：

$$\hat{P}_{d,t}^{L,(s)} = \hat{P}_{d,t}^L \text{clip}\left(\frac{\overline{P_{d,s-18:s}^L}}{\max(\hat{P}_{d,s-18:s}^L, 1)}, 0.75, 1.25\right), \quad t \geq s. \quad (26)$$

横线表示索引 $j = s - 18, \dots, s - 1$ 上的算术均值，分母中的 1 以 kW 计，用于避免预测均值过小导致比值异常。该比值反映最近实际负荷与原预测的偏差。按式(25)限制在 $[0.75, 1.25]$ 后，每次修正幅度至多为原预测的 25%；上下限由控制规则设定，未作置信水平解释。将修正后的负荷与融合光伏相减，形成净负荷点预测，再按式(15)加入对应发布时间的历史残差分位。每种 λ 和预报组合分别计算残差，所用样本均来自此前日期。

5.3.3 分条约束与汇总模型

a) 剩余时域供需平衡。仅对 $t \geq s$ 规划未来交付：

$$q_{d,t}^{(s)} + \hat{b}_{d,t} = \tilde{N}_{d,t}^{(s)} + \hat{c}_{d,t} + \hat{w}_{d,t}. \quad (27)$$

已交付区间不重新优化，其实际费用和库存变化均保留。

b) 实际库存起点。每次规划采用当时实际储电量，即 $\hat{S}_{d,s} = S_{d,s}$ ，并要求 $\hat{S}_{d,144} \geq 6000$ 。预测中的储能运行继续满足问题一的状态递推、功率和库存限制。

c) 原计划下界。若某段 $q < g$ ，可将 q 提高到 g ，差额计入未利用量。此时充放电和库存相同，还可省去减购违约费。因此，按主合约保留原计划量的费用更低，可加入

$$q_{d,t}^{(s)} \geq g_{d,t}, \quad t \geq s. \quad (28)$$

各次修订均以零时 g_t 为下界。此前的 $q_t^{(s-36)}$ 中尚未交付的追加量可以撤回，费用最终按交付量与原计划的净差计算。

d) 信息与动作时点。本次 $q^{(s)}$ 根据 $\mathcal{H}_{d,s}$ 制定；此后各交付段获得的实际需求用于充放电反馈和紧急补购，已交付电量按当时计划记录。

省略日期下标，得到每次日内重规划的完整形式：

$$\begin{aligned} (\mathbf{P}_3^{(s)}) \quad & \min \quad \sum_{t=s}^{143} 1.5p_t(q_t^{(s)} - g_t), \\ \text{s.t.} \quad & q_t^{(s)} + \hat{b}_t = \tilde{N}_t^{(s)} + \hat{c}_t + \hat{w}_t, \\ & \hat{S}_{t+1} = \hat{S}_t + 0.9\hat{c}_t - \hat{b}_t/0.9, \\ & 0 \leq \hat{c}_t, \hat{b}_t \leq 5000/6, \quad \hat{w}_t \geq 0, \quad q_t^{(s)} \geq g_t, \\ & 1200 \leq \hat{S}_t \leq 10800, \quad \hat{S}_s = S_{d,s}, \quad \hat{S}_{144} \geq 6000. \end{aligned} \quad (29)$$

区间约束作用于 $t = s, \dots, 143$ ，库存边界还包括 144。剩余时域分别为 108、72、36 段，相应线性规划为 540、360、180 个连续变量。充放电互斥仍可按问题一的论证处理。

5.3.4 求解与日内执行步骤

权重候选为 $\{0, 0.25, 0.50, 0.75, 1\}$ ，与六个分位组成 30 组参数。按 1 月 15—31 日实际费用比较，选得 $(\lambda, \alpha) = (0.50, 0.65)$ 。正式运行采用的 1 月热启动预先固定为纯附件三预测、0.80 分位及三次日内调整，2 月 1 日起使用所选参数。候选模拟仅用于评分，正式策略的初始库存由预先固定的热启动运行确定。

日前与日内协调优化已有相关研究^[2]。本问采用如下分时执行步骤：

- 1. 零时继承真实库存，读取已发布光伏预报，与历史曲线融合并作风险修正，求解全天原计划 g 。
- 2. 每到 06、12、18 点，接收本次新预报和此前已完成区间的负荷、光伏观测，更新剩余预测。
- 3. 以当前实际库存为起点，求解 $(P_3^{(s)})$ ，更新尚未交付的常规计划，并记录各次修订。
- 4. 在每个交付段以当前 q_t 替换问题二反馈式中的 g_t ，执行储能和必要的紧急购电。
- 5. 根据最终 $q_t - g_t$ 计算主合约调整费，日末将实际库存传至次日。

日内 LP 给出当次预测下费用最低的常规计划，后续效果还取决于实际需求和储能反馈。因此，本问得到的是基于预测的可行调度策略。

5.3.5 调度结果与费用来源

正式期总费用为 13,315,835.99 元，比问题二减少 592,339.96 元，降幅 4.26%。表9将费用、电量和库存在同一口径下比较。

表 9 固定电价下两种策略的结果（2—12 月）

指标	零时固定计划	日内调整计划
原计划费/元	13,291,625.72	12,649,467.17
调整费/元	0.00	527,937.93
紧急费/元	616,550.23	138,430.89
总费用/元	13,908,175.95	13,315,835.99
常规购电/kWh	21,712,548.31	21,197,074.02
紧急购电/kWh	102,787.48	25,862.57
未利用电量/kWh	2,646,069.43	2,021,445.81
发生紧急购电的天数	115	139
期初储电量/kWh	6,840.33	7,089.87
期末储电量/kWh	6,258.37	5,987.72

原计划费由 13291625.72 元降至 12649467.17 元，紧急费由 616550.23 元降至 138430.89 元，两项节省超过了新增的 527937.93 元调整费。常规购电量和未利用电量

也有所下降，日内方案通过重新安排购电数量和时段，减少了提前多购与高价补购的支出。

紧急购电总量下降约 74.84%，发生天数却从 115 天增至 139 天，说明日内方案降低了紧急电量及支出，发生频率仍需关注。两种主策略的热启动不同，表中同时列出首末库存；费用差反映各自连续运行的实际支出。

表10汇总四个指定日期，正文按题面格式列出 9 月 23 日，其他日期及紧急事件、账单见附录A.2，与 result3.xlsx 对应。购电表用相同的六列结构分别展示零时原计划和最终常规购电，两者之差才是净调整量。原计划部分的“全天购电费”为原计划费，最终部分为原计划费加调整费，均不含紧急费；储能表为实际执行值。

表 10 问题三的指定日期结果（2025 年）

日期	常规购电/kWh	紧急购电/kWh	全天总费用/元
03-20	67,646.32	0.00	41,647.19
06-21	35,854.93	0.00	20,909.49
09-23	67,230.48	170.85	43,904.67
12-21	96,803.32	0.00	62,129.52

2025.9.23 的购电和储能结果分别见表11和表12。

表 11 微网在指定时间段的购电量及全天的购电量和购电费（2025.9.23）

零时原计划					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	401.83	14:00–14:10	0.00
16:00–16:10	520.87	18:00–18:10	763.88	20:00–20:10	0.00
全天购电量	65,285.63	全天购电费	40,305.24		
最终常规购电					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	401.83	14:00–14:10	0.00
16:00–16:10	520.87	18:00–18:10	793.71	20:00–20:10	0.00
全天购电量	67,230.48	全天购电费	43,084.92		

表 12 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.9.23）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4,254.93	310.26	4:00–8:00	1,030.38	6,166.49
8:00–12:00	5,281.76	1,896.62	12:00–16:00	5,321.25	534.56
16:00–20:00	111.95	5,842.00	20:00–24:00	5,560.36	2,696.82
0:00 储电量		6,177.67	24:00 储电量		6,196.96

9 月 23 日紧急电量从问题二的 892.40 kWh 降至 170.85 kWh，总费下降 3909.92 元；3 月 20 日日内方案却比问题二多支出 109.40 元，说明调整机会并不保证每一天都降低费用。图3展示 9 月 23 日的原计划、最终购电与实际净需求；竖虚线标出三次允许修订的时刻。最终购电在原计划上补充部分缺口，后续库存因此改变，紧急购电的规模随之下降。

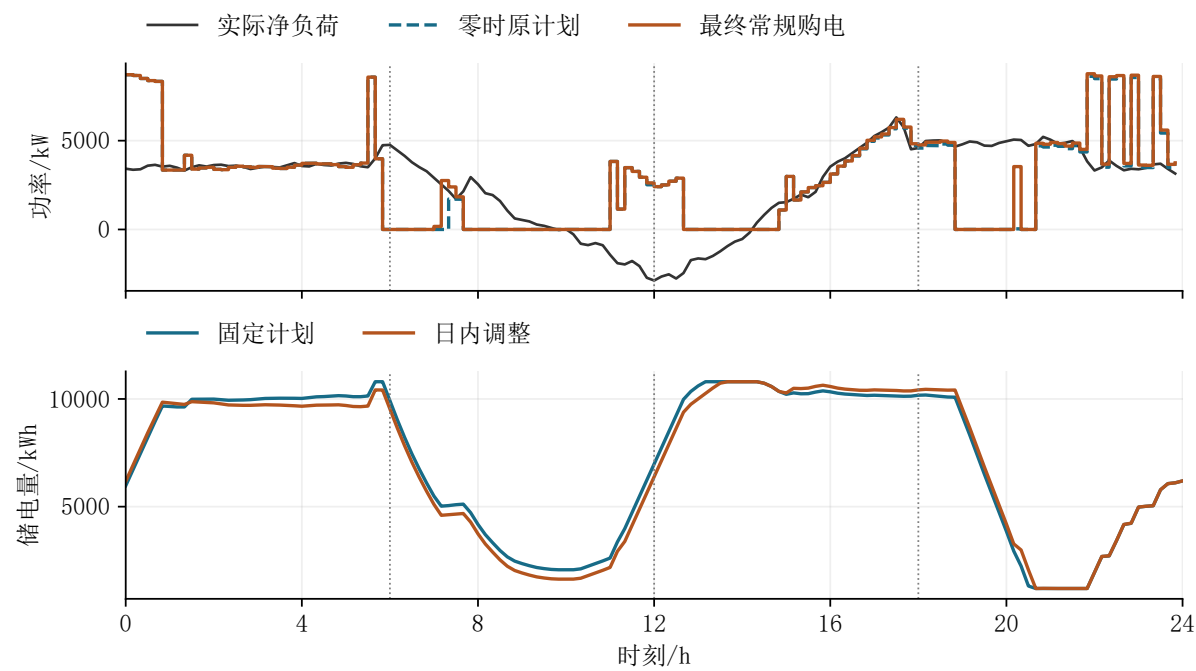


图 3 9 月 23 日的日内购电调整与库存变化

5.3.6 预报贡献与参数灵敏度

各光伏来源先在相同 1 月区间选参，再从共同热启动和期初库存开始比较。融合方案比纯历史基准节省 112379.91 元，单用附件三则更贵，说明保留历史预测有助于降低本题的运行费用。完整费用见附录B.1表58。某次新预报的作用，还需进一步控制调整机会等条件。

为判断是否需要其他时刻的预报，各组采用相同主参数、三次调整时刻、负荷修正及光伏插值起点，逐组改变允许读取的新预报；不读取时沿用最近可用的一次。各组共

用 1 月热启动，随后按自身调度连续运行。表13列出每次单独撤去一项预报后的费用变化，八种组合的完整结果见附录B.1表56。

表 13 单独撤去一次新小时预报的条件费用差（元）

撤去的预报	纯附件 3	融合主策略
06 点	53,593.3113	28,402.5518
12 点	74,628.9683	20,127.5241
18 点	-0.1296	-0.3123

读取全部新预报比均不读取节省 17225.82 元。撤去 06 或 12 点的新预报会增加费用，撤去 18 点后的变化仅 -0.31 元。因此，在本组参数下，前两次新预报有助于降低费用，18 点的新预报收益很小。各次预报的影响随库存和后续计划相互关联，单项费用差不能直接相加。对照仍保留 18 点调整，可利用当时的负荷观测和库存变化重新安排购电。

分位扰动见表14。正式期 $\alpha = 0.50$ 的费用比主参数 0.65 低 15787.50 元，但紧急电量明显增大。1 月评分较低的参数未必在后续月份同样省费，短期选参存在样本局限。主结果采用按 1 月费用选出的参数，全年扰动用于考察这一选择的影响。

表 14 问题三的分位参数扰动（2—12 月）

α	总费用/元	紧急购电/kWh	年末储电量/kWh
0.5	13,300,048.49	66,633.50	5,890.36
0.65	13,315,835.99	25,862.57	5,987.72
0.7	13,393,753.32	18,804.41	6,060.07
0.9	14,101,446.70	2,544.90	6,507.64

将日末预测恢复目标改为 3000 或 9000 kWh，结果见附录B.1表57。目标较低时支出下降，热启动库存和期末库存也随之降低，因此费用变化同时反映了购电安排和储备电量的差异。

逐笔成交不可撤回时，重新选参和优化的总费为 13,460,502.62 元，仍比问题二低 447673.33 元，但节省少于主合约。附录B.2分别给出原调度重新计费及重新优化的结果，说明累计调整费的规则会影响方案收益。

5.4 问题四：波动电价下的规划

5.4.1 价格信息与两种决策频率

问题四采用附件四的实时波动电价。价格变化会影响哪些时段适合购电和储能，因此需要根据预测价格重新制定计划，再按实际价格计算支出。未来电价在决策时尚未知

晓，预测控制与预先知道全部价格时的表现存在差别，相关动态价格研究也讨论了这一问题^[4]。

本文将“交易时刻电价”解释为对应交付区间的实际价格 $p_{d,t}$ ，在交付和结算时使用。沿用问题三的交易变量，记 $\hat{p}_{d,t}^{(s)}$ 为时刻 s 根据已有信息预测的未来价格。问题 4-2 在 $s = 0$ 制定全天固定计划；问题 4-3 在 $s \in \{0, 36, 72, 108\}$ 规划，并沿用光伏预报融合及最终净调整合约。

两种策略采用相同设备参数、十分钟步长和充放电反馈，实际库存均随每日运行连续传递。

5.4.2 费用目标与价格预测

问题 4-2 和 4-3 的实际购电费用分别为

$$C_{4-2} = \sum_{d,t} p_{d,t} (g_{d,t} + 5e_{d,t}), \quad (30)$$

$$C_{4-3} = \sum_{d,t} p_{d,t} [g_{d,t} + 1.5(q_{d,t} - g_{d,t})_+ + 0.5(g_{d,t} - q_{d,t})_+ + 5e_{d,t}]. \quad (31)$$

费用均统计 2—12 月，并按实际价格计算。制定计划时，用预测价格代替尚未知晓的未来价格，零时和日内分别求解

$$\min \sum_{t=0}^{143} \hat{p}_{d,t}^{(0)} g_{d,t}, \quad \min \sum_{t=s}^{143} 1.5 \hat{p}_{d,t}^{(s)} (q_{d,t}^{(s)} - g_{d,t}). \quad (32)$$

第二个目标用于问题 4-3 的 $s > 0$ 。两项目标决定当次购电安排，最终费用则由实际交付电量和价格核算。

价格预测同样利用每周规律。对 $d \geq 7$ ，取此前与目标日星期相同的日期，对同刻价格加权，得到

$$\bar{p}_{d,t} = \frac{\sum_{k=1}^{K_d} 0.55^{k-1} p_{d-7k,t}}{\sum_{k=1}^{K_d} 0.55^{k-1}}, \quad K_d = \min\{4, \lfloor d/7 \rfloor\}. \quad (33)$$

历史不足一周时，取此前最多 7 天，以 0.8 的衰减系数加权。零时取 $\hat{p}_{d,t}^{(0)} = \max(\bar{p}_{d,t}, 0.001)$ 。问题 4-3 在日内根据最近 18 段实际价格的均值修正预测水平，使用式(25)将修正比例限制在 $[0.5, 1.5]$ 。记

$$r_{d,s}^p = \text{clip} \left(\frac{\frac{1}{18} \sum_{j=s-18}^{s-1} p_{d,j}}{\max \left(\frac{1}{18} \sum_{j=s-18}^{s-1} \bar{p}_{d,j}, 0.01 \right)}, 0.5, 1.5 \right), \quad (34)$$

则剩余预测为

$$\hat{p}_{d,t}^{(s)} = \max\{0.001, r_{d,s}^p \bar{p}_{d,t}\}, \quad t \geq s, \quad d \geq 1. \quad (35)$$

比例截断可避免历史均值很小时产生过大的修正，价格下限保证目标系数为正。首日各发布时间均采用附件一的固定价格，用于1月初始化；从随后日期起，按已完成区间的观测计算预测和修正比例。

5.4.3 分条约束与汇总模型

a) 物理约束。每次规划仍满足风险净负荷下的能量平衡、储能效率、功率和安全库存限制。价格改变目标系数，不改变物理守恒关系。

b) 库存衔接。零时从实际 $S_{d,0}$ 开始，日内从实际 $S_{d,s}$ 开始，预测日末恢复目标均为 6000 kWh。实际跨日运行不重置库存。

c) 交易频率与原计划下界。问题 4-2 全天 $q_t = g_t$ ；问题 4-3 在三次允许时刻修改未交付量，且 $q_t^{(s)} \geq g_t$ 。实际和预测价格均为正，故问题三关于保留原计划量更有利的论证仍适用。

d) 可用信息与实际结算。预测价格、净负荷和本次计划均由 $\mathcal{H}_{d,s}$ 确定；之后观测到的 $p_{d,t}$ 用于结算对应交付段的费用，已执行决策保留原记录。

为统一两种频率，令允许的发布时间集合为 $\mathcal{S}$ ；零时取 $\ell_t^{(0)} = 0$ 、 $\kappa_0 = 1$ ，日内取 $\ell_t^{(s)} = g_t$ 、 $\kappa_s = 1.5$ 。每次求解

$$\begin{aligned} (\mathbf{P}_4^{(s)}) \quad & \min \quad \sum_{t=s}^{143} \kappa_s \hat{p}_{d,t}^{(s)} (q_t^{(s)} - \ell_t^{(s)}), \\ \text{s.t.} \quad & q_t^{(s)} + \hat{b}_t = \tilde{N}_{d,t}^{(s)} + \hat{c}_t + \hat{w}_t, \\ & \hat{S}_{t+1} = \hat{S}_t + 0.9\hat{c}_t - \hat{b}_t/0.9, \\ & 0 \leq \hat{c}_t, \hat{b}_t \leq 5000/6, \quad q_t^{(s)} \geq \ell_t^{(s)}, \quad \hat{w}_t \geq 0, \\ & 1200 \leq \hat{S}_t \leq 10800, \quad \hat{S}_s = S_{d,s}, \quad \hat{S}_{144} \geq 6000. \end{aligned} \quad (36)$$

零时求得的计划为 $g_t = q_t^{(0)}$ 。问题 4-2 取 $\mathcal{S} = \{0\}$ ，问题 4-3 取 $\mathcal{S} = \{0, 36, 72, 108\}$ ；区间约束作用于 $t = s, \dots, 143$ ，库存边界还包括 144。模型据此统一表示两种规划频率。

5.4.4 求解与结算步骤

沿用问题二、三的候选集合，按 1 月 15—31 日实际费用选参。问题 4-2 选得 $\alpha = 0.80$ ，问题 4-3 选得 $(\lambda, \alpha) = (0.50, 0.65)$ 。问题 4-3 的 1 月热启动固定使用纯附件三预测、0.80 分位及三次调整，2 月起采用所选参数。

按时间先后运行以下步骤：

1. 零时继承真实库存，以过去日期分别预测负荷、光伏和价格；构造风险净负荷。
2. 用预测价格作为目标系数求解 $(\mathbf{P}_4^{(0)})$ ，锁定原计划并保存预测曲线。

3. 问题 4-2 按固定计划运行；问题 4-3 在各发布时间利用新预报及最近观测更新负荷、光伏和价格预测，求解剩余时段模型。
 4. 交付时根据当段实际净负荷执行充放电，放电后的缺口由紧急购电补足。
 5. 用该段实际价格结算原计划、最终净调整和紧急购电，更新实际库存。
 6. 日末传递库存，2 月起累计正式费用；从逐段数组分别重算全年和指定日期账单。
- 为检查信息使用顺序，将未来实际价格替换后重新计算，确认当前预测和计划不受影响。历史误差也按日期和发布时间读取，只有已完成区间的误差参与当前修正。

5.4.5 波动电价结果及比较

问题 4-2 总费用为 14,675,844.87 元，问题 4-3 为 14,044,885.25 元，日内方案节省 630,959.63 元，降幅 4.30%。表15给出费用与电量分解。

表 15 波动电价下两种策略的结果（2—12 月）

指标	问题 4-2	问题 4-3
原计划费/元	14,032,328.84	13,366,608.69
调整费/元	0.00	550,605.94
紧急费/元	643,516.03	127,670.62
总费用/元	14,675,844.87	14,044,885.25
常规购电/kWh	21,708,087.29	21,192,613.91
紧急购电/kWh	101,425.76	24,227.41
未利用电量/kWh	2,632,806.82	2,009,834.69
发生紧急购电的天数	108	134
期初储电量/kWh	6,864.27	7,114.54
期末储电量/kWh	6,263.02	5,997.11

日内方案降低了原计划费和紧急费，计入新增 550605.94 元调整费后，总支出仍较低。紧急电量由 101425.76 kWh 降至 24227.41 kWh，发生天数却由 108 增至 134。与固定电价下相同，紧急购电的总规模缩小，但分布在更多日期。

图4给出逐月费用及紧急电量，便于在相同月份和实际电价下比较两种策略。固定电价与波动电价的费用差还包含价格水平、购电安排和库存变化，解释这一差异时需同时考虑这些因素。

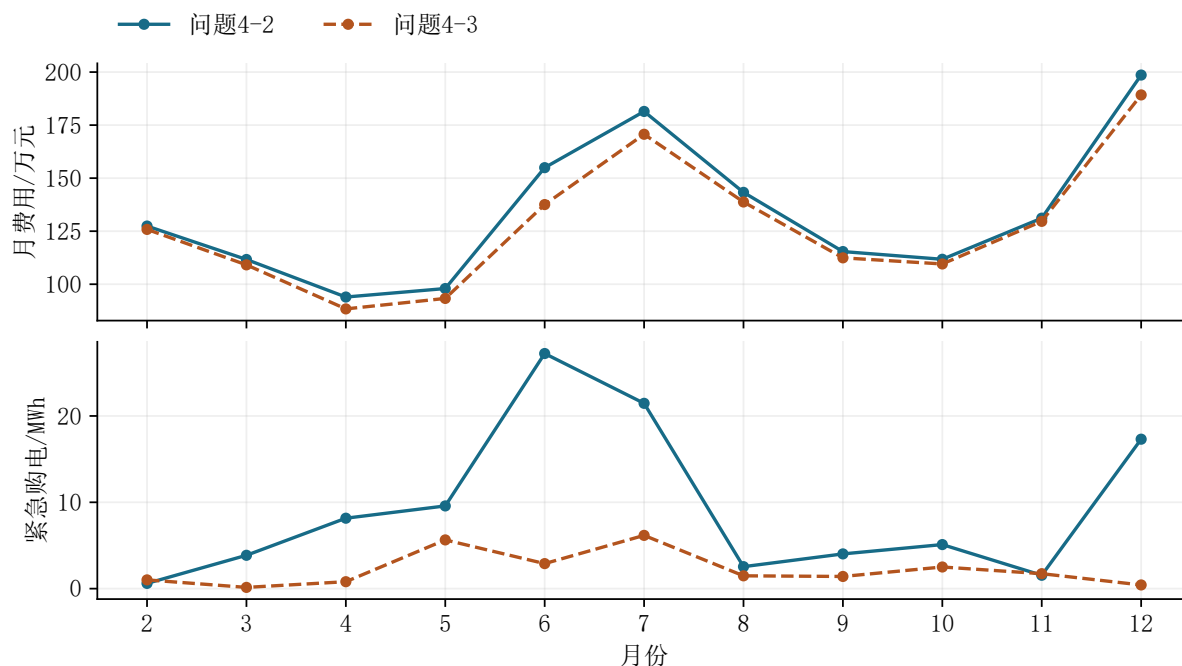


图 4 波动电价下各月费用与紧急购电量

指定日期费用列于表16。在 9 月 23 日，允许调整后的全天总费从 49762.23 元降至 45799.34 元；12 月 21 日两种方案费用均较高。下一小节分别展示 9 月 23 日的购电与储能；其余日期、紧急事件及账单见附录A.3、A.4，完整策略保存在 result4-2.xlsx、result4-3.xlsx。

表 16 波动电价下指定日期费用（2025 年，元）

日期	问题 4-2	问题 4-3	日内方案节省
03-20	42,627.22	42,600.22	27.00
06-21	19,640.47	18,336.05	1,304.42
09-23	49,762.23	45,799.34	3,962.89
12-21	75,769.21	72,885.14	2,884.07

在共同热启动下比较光伏来源，融合方案比纯历史基准少支出 122075.92 元，也比单用附件三更省费，完整结果见附录B.1表59。各预测方法分别选择参数，因此费用差反映了预测方法与相应控制参数共同作用的结果。

5.4.6 代表日期的购电与储能结果

a) 问题 4-2：零时固定计划。以下按题面表 1、表 2 列出 9 月 23 日。购电表给出常规计划电量及按实际交付价格结算的常规费；电量为 kWh，费用为元。2025.9.23 的购电和储能结果分别见表17和表18。

表 17 微网在指定时间段的购电量及全天的购电量和购电费（2025.9.23）

时间段	购电量	时间段	购电量	时间段	购电量	
10:00–10:10	0.00	12:00–12:10	499.87	14:00–14:10	0.00	
16:00–16:10	531.67	18:00–18:10	775.50	20:00–20:10	0.00	
全天购电量		68,171.89		全天购电费		44,011.52

表 18 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.9.23）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4,837.74	54.89	4:00–8:00	801.19	6,062.63
8:00–12:00	5,459.91	1,896.62	12:00–16:00	4,418.80	560.16
16:00–20:00	79.59	5,673.19	20:00–24:00	5,585.42	2,617.77
0:00 储电量		5,901.66	24:00 储电量		6,226.87

b) 问题 4-3：日内调整。购电表分别展示零时原计划与最终常规购电，费用口径与问题三相同，均按实际交付价格核算；储能表列实际运行结果。指定日的初始库存承接前日，两组方案均从全年起点 6000 kWh 连续运行至该日。2025.9.23 的购电和储能结果分别见表19和表20。

表 19 微网在指定时间段的购电量及全天的购电量和购电费（2025.9.23）

零时原计划					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	401.83	14:00–14:10	0.00
16:00–16:10	520.87	18:00–18:10	763.88	20:00–20:10	0.00
全天购电量		65,294.82		全天购电费	
				42,202.79	
最终常规购电					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	401.83	14:00–14:10	0.00
16:00–16:10	520.87	18:00–18:10	793.71	20:00–20:10	0.00
全天购电量		67,099.79		全天购电费	
				44,903.59	

表 20 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.9.23）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4,267.49	194.62	4:00–8:00	996.54	6,251.03
8:00–12:00	5,281.76	1,896.62	12:00–16:00	5,288.36	509.61
16:00–20:00	111.95	5,255.66	20:00–24:00	5,591.15	3,287.90
0:00 储电量		6,169.40	24:00 储电量		6,224.67

5.4.7 价格与参数灵敏度检验

先检验预测峰谷价差的影响。记 $m_{d,s}$ 为本次剩余时段的预测价格均值，将价格曲线改为

$$\hat{p}_{d,t}^{(s,\theta)} = \max\{0.001, m_{d,s} + (1 + \theta)(\hat{p}_{d,t}^{(s)} - m_{d,s})\}, \quad \theta \in \{-0.2, 0, 0.2\}. \quad (37)$$

负扰动缩小预测峰谷差，正扰动扩大预测峰谷差，均根据当时可得的预测曲线计算。各组采用相同负荷、光伏、实际结算价、控制参数及 1 月热启动，从 2 月起按扰动后的预测重新规划并连续运行。表 21 列出实际费用和年末库存。

表 21 预测价差幅度扰动（共同热启动，2—12 月）

方案	θ	总费用/元	较主方案增费/元	期末储电量/kWh
问题 4-2	-0.2	14,674,479.32	-1,365.55	6,263.02
问题 4-2	0	14,675,844.87	0.00	6,263.02
问题 4-2	+0.2	14,683,290.13	7,445.26	6,263.02
问题 4-3	-0.2	14,042,823.76	-2,061.48	5,997.11
问题 4-3	0	14,044,885.25	0.00	5,997.11
问题 4-3	+0.2	14,061,149.66	16,264.42	5,997.11

扩大预测价差 20% 时，两种策略分别增费 7445.26 元和 16264.42 元；缩小 20% 时分别减费 1365.55 元和 2061.48 元。四组扰动与各自主方案的期末库存相同，可在相同储备水平下比较支出。本次范围内总费变化均小于 0.12%，日内调整方案仍更省费。该检验说明策略对价差幅度的小范围变化较稳定，峰谷时刻错位和极端电价的影响仍待检验。

另作已知未来价格的对照，直接用实际价格规划，问题 4-2 与 4-3 费用分别为 14,566,416.06 元和 13,932,754.85 元。该对照使用了执行时无法提前获得的信息，热启动和后续库存也随之变化，因此仅作为事后参考，尚不足以给出随机全年问题的严格下界。实际预测策略的零时平均绝对价格误差约为 0.04456 元/kWh，其费用影响需结合购

电量和储能动作分析。

风险参数的选择结果见附录B.1表60。按1月15—31日费用评分，问题4-3固定融合权重0.50，固定计划和日内调整分别在0.80和0.65处取得候选中的最低费用。提高分位可减少缺口，也会增加提前购电支出；这里的选择仅依据1月评分。

跨月实验按训练先于评价的顺序^[11]，每月根据上月费用重选参数，各组从共同4月1日库存连续运行。问题4-3的逐月选参方案比固定参数多支出16320.66元，9个月中仅1个月更省，说明频繁重选在本年度没有改善累计费用。完整账单与库存见附录B.4。

在相同信息、参数及热启动条件下，将储能控制换成确定性滚动反馈^[3]，问题4-2和4-3分别增费33995.79元、213489.58元，期末库存与原反馈相同。该配置尚未估计跨日库存价值，结果说明它在本组设置下未优于原反馈；进一步改进需同时考虑次日的用电需求。模型及费用分解见附录B.3。

六、 结果分析与模型检验

6.1 主策略费用汇总

四种主策略在2025年2—12月334天内的费用见表22，原计划费、调整费和紧急费分别统计后相加。问题三比问题二节省4.26%，问题4-3比问题4-2节省4.30%，两种电价条件下日内调整均降低了总支出。固定电价与波动电价之间的差额，还受到价格水平和购电安排变化的影响。

表 22 四种主方案的正式期费用（2025 年 2—12 月，元）

方案	原计划费用	调整费用	紧急费用	总费用
问题 2	13,291,625.72	0.00	616,550.23	13,908,175.95
问题 3	12,649,467.17	527,937.93	138,430.89	13,315,835.99
问题 4-2	14,032,328.84	0.00	643,516.03	14,675,844.87
问题 4-3	13,366,608.69	550,605.94	127,670.62	14,044,885.25

6.2 物理、信息与账单核验

为检查实际调度是否满足模型约束，根据每个交付区间记录的电量和储电量，独立计算供需平衡残差与储能递推残差：

$$\begin{aligned}
 r_{d,t}^E &= q_{d,t} + e_{d,t} + V_{d,t} + b_{d,t} - L_{d,t} - c_{d,t} - w_{d,t}, \\
 r_{d,t}^S &= S_{d,t+1} - S_{d,t} - 0.9c_{d,t} + b_{d,t}/0.9.
 \end{aligned} \tag{38}$$

两类残差均在 10^{-6} kWh 容差内。逐段检查还确认了库存和功率满足边界、充放电互斥，以及日末库存与次日日初相同。原计划、最终调整和紧急购电费用分别重算后，全年及指定日期账单均与逐段汇总一致。

信息检验分别替换未来实测数据和未发布的预报，确认当前预测与计划保持相同；再改变已获得的信息，检查方法能否作出相应更新。对各主方案、合约与反馈对照、跨月实验以及参数扰动分别核验，均满足物理约束、信息使用顺序和费用一致的要求。问题一的最优性还由前述 LP 与 MILP 结果支持。

6.3 补充对照及结论边界

逐笔收费、价格感知反馈及上月选参的完整账单分别见附录B.2、B.3、B.4。逐笔制度下重新优化，固定与波动电价的日内方案分别节省 447673.33 元和 475880.51 元，节省额均低于主合约。四组滚动反馈增加的紧急费超过了常规费节省；逐月选参的融合方案在问题三、4-3 中也分别多支出 2109.14 元和 16320.66 元。合约规则会改变收益，增加控制和选参频率也未必降低费用。以上均为同年数据上的比较，其他年度仍需按固定的方法和选参规则另行验证。

七、 模型的评价与推广

7.1 模型的优点和不足

模型把购电安排与储能运行联系起来，能够解释费用为何变化。低价时段可以多购电，但充电量受功率和容量限制；需求超过计划时，电池先补足缺口，剩余部分才产生紧急费用。每个十分钟时段都记录实际电量、库存和支出，因此既能检查调度是否可行，也能从账单中判断节省来自常规购电还是紧急补购。允许不利用余电后，规划可用线性模型求解，问题一还通过整数模型确认了最低费用。

各次决策只读取当时已获得观测和预报，实际储电量也随运行连续传递，这使全年模拟符合题目给定的信息条件。问题三进一步保持调整时刻相同，比较读取不同预报时的费用，因而能够辨别新增预报的作用。这项对照尤其说明，18 时的新预报收益很小，但该时刻仍可根据库存和负荷变化调整购电。

当前方法的主要局限在于风险和跨日库存的处理。历史误差分位逐段修正净负荷，尚未描述连续多段低估需求的风险；实时反馈优先满足眼前缺口，也没有估计把电留到次日的价值。日末预测库存目标只能部分缓解这一问题。所测试的价格感知反馈增加了紧急费用，说明后续控制设计需要同时考虑预测误差和跨日用电需求。

参数选择受样本影响。问题三按 1 月费用选出的风险分位，全年支出高于另一候选，逐月重选也未降低累计费用，反映了短期选参的局限。现有结果仍需其他年度检验。实际运行还受设备条件限制：本文假定紧急购电能补足缺口，尚未计入电池老化、外网容量和响应延迟，这些因素均会影响供电能力和成本。

7.2 模型的推广

将模型用于其他微网时，首先需要按当地的观测和预报发布时间整理负荷、光伏与电价数据，并核实购电合约是否允许撤回尚未交付的追加量。随后固定预测方法和选参规则，在未参与方法开发的年度连续运行，比较实际费用、紧急电量及期初期末库存，才能判断本题中的节省是否能够延续。

实际部署还需补充设备和电网约束。电池资料可用于估计每次充放电的折损成本，以及老化后可用容量的变化；外网供电上限则应同时约束常规和紧急购电。若仍有无法满足的需求，需在模型中记录缺供量及相应代价。研究扩容方案时，还要将购电节省与设备投资、寿命成本一并比较。

针对连续预测偏差，可构造保留误差时间相关性的多时段场景，将规划延长到次日，并估计期末剩余电量的价值。多时间尺度控制和分布鲁棒优化研究^[5, 12]为此提供了参考。各方案应在相同信息、合约和初始库存下比较费用与计算时间。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于数据处理与建模分析、代码生成与调试、文献检索与整理、计算结果核对、论文撰写与排版，详细使用情况见支撑材料。

参考文献

- [1] 吴杰, 温晨阳, 李珊, 等. 基于分时电价的光伏-储能系统容量优化配置[J/OL]. 电工电能新技术, 2018, 37 (1): 23-30. <https://ateee.iee.ac.cn/CN/abstract/abstract1702.shtml>. DOI:10.12067/ATEEE1704027.
- [2] 肖海伟, 孙久严, 李章溢, 等. 基于综合储能设备的工业园区联络线功率实时 MPC 控制方法[J/OL]. 电力建设, 2019, 40 (8): 34-42. <https://www.cepc.com.cn/CN/10.3969/j.issn.1000-7229.2019.08.005>.
- [3] Wytock M, Moehle N, Boyd S. Dynamic energy management with scenario-based robust MPC[C/OL]//2017 american control conference (ACC) . Seattle, WA, USA: IEEE, 2017: 2042-2047. https://web.stanford.edu/~boyd/papers/pdf/dyn_ener_man_acc.pdf.
- [4] Perez-Pineiro D, Skogestad S, Boyd S. Home energy management with dynamic tariffs and tiered peak power charges [Manuscript][EB/OL]. (2023) [2026-09-10]. <https://stanford.edu/~boyd/papers/hem.html>.
- [5] 李嘉伟, 巨云涛, 张璐, 等. 基于分布鲁棒模型预测控制的微电网多时间尺度优化调度[J/OL]. 电力工程技术, 2024, 43 (4): 45-55. <https://epet-info.com/html/2024/4/230524600.html>. DOI:10.12158/j.2096-3203.2024.04.005.

- [6] The SciPy community. linprog (method='highs') : SciPy reference guide[EB/OL]. [2026-09-10]. <https://docs.scipy.org/doc/scipy/reference/optimize.linprog-highs.html>.
- [7] Huangfu Q, Hall J A J. Parallelizing the dual revised simplex method[J/OL]. Mathematical Programming Computation, 2018, 10 (1): 119-142. <https://link.springer.com/article/10.1007/s12532-017-0130-5>.
- [8] 胡俊杰, 曲佳桐, 刘雪涛, 等. 考虑应急电源车的高速公路服务区微网两阶段优化调度方法[J/OL]. 电力建设, 2025, 46 (4): 150-159. <https://www.cepc.com.cn/CN/10.12204/j.issn.1000-7229.2025.04.013>.
- [9] Bates J M, Granger C W J. The combination of forecasts[J/OL]. Journal of the Operational Research Society, 1969, 20 (4): 451-468. <https://www.tandfonline.com/doi/abs/10.1057/jors.1969.103>.
- [10] 隗知初, 杨苹, 周钱雨凡, 等. 基于 Q-Learning 的多模态自适应光伏功率优化组合预测[J/OL]. 电力工程技术, 2026, 45 (1): 115-124,163. <https://epet-info.com/html/2026/1/20260110.html>. DOI:10.12158/j.2096-3203.2026.01.011.
- [11] Cerqueira V, Torgo L, Mozetič I. Evaluating time series forecasting models: an empirical study on performance estimation methods[J/OL]. Machine Learning, 2020, 109 (11): 1997-2028. <https://link.springer.com/article/10.1007/s10994-020-05910-7>.
- [12] 李佳玮, 孙庆贺, 王琼, 等. 计及不确定相关性的多能微电网数据驱动分布鲁棒优化调度[J/OL]. 电力建设, 2025, 46 (8): 22-33. <https://www.cepc.com.cn/CN/10.12204/j.issn.1000-7229.2025.08.003>.

A 指定日期的补充结果

正文保留四日期汇总及 9 月 23 日购电、储能原格式表。本附录列出其余三个日期，并按题面表 3 并列四日期全部紧急事件，另列完整账单；电量为 kWh、费用为元，计量与舍入口径见第 5.2 节。

A.1 问题二

9 月 23 日购电与储能见正文表 6、7。2025.3.20 的购电和储能结果分别见表 23 和表 24。

表 23 微网在指定时间段的购电量及全天的购电量和购电费（2025.3.20）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	605.05	14:00–14:10	0.00
16:00–16:10	493.66	18:00–18:10	711.53	20:00–20:10	0.00
全天购电量		68,386.72		全天购电费	
				41,537.79	

表 24 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.3.20）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	3,932.12	143.66	4:00–8:00	1,068.89	5,197.90
8:00–12:00	6,214.23	2,777.38	12:00–16:00	3,718.36	415.81
16:00–20:00	261.15	5,734.03	20:00–24:00	5,662.99	2,950.27
0:00 储电量		6,663.82	24:00 储电量		6,303.52

2025.6.21 的购电和储能结果分别见表 25 和表 26。

表 25 微网在指定时间段的购电量及全天的购电量和购电费（2025.6.21）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	247.19	18:00–18:10	0.00	20:00–20:10	0.00
全天购电量		37,707.19		全天购电费	
				22,414.49	

表 26 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量 (2025.6.21)

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	1,016.02	1,120.01	4:00–8:00	437.87	4,390.39
8:00–12:00	8,456.58	0.00	12:00–16:00	0.00	0.00
16:00–20:00	312.75	5,157.72	20:00–24:00	6,563.91	2,485.90
0:00 储电量		8,003.24	24:00 储电量		8,496.09

2025.12.21 的购电和储能结果分别见表27和表28。

表 27 微网在指定时间段的购电量及全天的购电量和购电费 (2025.12.21)

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	994.23	14:00–14:10	0.00
16:00–16:10	855.14	18:00–18:10	733.64	20:00–20:10	0.00
全天购电量		99,788.74	全天购电费		64,446.61

表 28 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量 (2025.12.21)

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4,514.59	57.06	4:00–8:00	910.22	1,414.62
8:00–12:00	5,914.67	6,794.16	12:00–16:00	6,186.24	2,220.11
16:00–20:00	0.00	5,577.31	20:00–24:00	5,753.40	2,842.53
0:00 储电量		6,677.89	24:00 储电量		6,622.69

表29按题面表 3 横向列出四个日期的全部紧急购电事件，表30给出含紧急费的完整账单。

表 29 微网在指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
无	0.00	无	0.00	20:20–21:50	892.40	无	0.00
全天合计	0.00	全天合计	0.00	全天合计	892.40	全天合计	0.00

表 30 问题 2 指定日期的全天账单（元）

日期	原计划费	调整费	紧急费	总费用
03-20	41,537.79	0.00	0.00	41,537.79
06-21	22,414.49	0.00	0.00	22,414.49
09-23	42,079.63	0.00	5,734.95	47,814.59
12-21	64,446.61	0.00	0.00	64,446.61

A.2 问题三

9 月 23 日购电与储能见正文表11、12。购电表分别列零时原计划与最终常规购电，后者费用含调整费。2025.3.20 的购电和储能结果分别见表31和表32。

表 31 微网在指定时间段的购电量及全天的购电量和购电费（2025.3.20）

零时原计划					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	557.73	14:00–14:10	0.00
16:00–16:10	553.12	18:00–18:10	708.74	20:00–20:10	0.00
全天购电量		66,614.07	全天购电费		40,394.87
最终常规购电					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	557.73	14:00–14:10	0.00
16:00–16:10	553.12	18:00–18:10	708.74	20:00–20:10	0.00
全天购电量		67,646.32	全天购电费		41,647.19

表 32 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.3.20）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	3,934.61	284.62	4:00–8:00	1,085.82	5,512.63
8:00–12:00	5,764.68	2,777.38	12:00–16:00	4,814.41	262.75
16:00–20:00	26.19	5,714.39	20:00–24:00	5,567.69	2,988.71
0:00 储电量		6,570.81	24:00 储电量		6,155.47

2025.6.21 的购电和储能结果分别见表33和表34。

表 33 微网在指定时间段的购电量及全天的购电量和购电费（2025.6.21）

零时原计划					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	204.64	18:00–18:10	0.00	20:00–20:10	0.00
全天购电量		35,822.67	全天购电费		20,860.00
最终常规购电					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	204.64	18:00–18:10	0.00	20:00–20:10	0.00
全天购电量		35,854.93	全天购电费		20,909.49

表 34 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.6.21）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	614.20	920.51	4:00–8:00	421.40	4,390.39
8:00–12:00	9,383.47	0.00	12:00–16:00	0.00	0.00
16:00–20:00	158.82	5,371.22	20:00–24:00	6,203.59	2,485.90
0:00 储电量		7,323.84	24:00 储电量		7,796.04

2025.12.21 的购电和储能结果分别见表35和表36。

表 35 微网在指定时间段的购电量及全天的购电量和购电费（2025.12.21）

零时原计划					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	959.50	14:00–14:10	0.00
16:00–16:10	849.15	18:00–18:10	723.11	20:00–20:10	0.00
全天购电量		96,792.11	全天购电费		62,109.93
最终常规购电					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	959.50	14:00–14:10	0.00
16:00–16:10	849.15	18:00–18:10	723.11	20:00–20:10	0.00
全天购电量		96,803.32	全天购电费		62,129.52

表 36 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.12.21）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4,565.52	131.98	4:00–8:00	1,321.73	1,481.00
8:00–12:00	5,703.78	7,129.08	12:00–16:00	6,932.53	2,283.94
16:00–20:00	10.53	5,708.00	20:00–24:00	5,587.69	2,843.50
0:00 储电量		6,379.90	24:00 储电量		6,336.73

表37按题面表 3 横向列出四个日期的全部紧急购电事件，表38给出含紧急费的完整账单。

表 37 微网在指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
无	0.00	无	0.00	20:20–20:40	15.16	无	0.00
				20:50–21:10	90.07		
				21:20–21:50	65.62		
全天合计	0.00	全天合计	0.00	全天合计	170.85	全天合计	0.00

表 38 问题 3 指定日期的全天账单（元）

日期	原计划费	调整费	紧急费	总费用
03-20	40,394.87	1,252.32	0.00	41,647.19
06-21	20,860.00	49.49	0.00	20,909.49
09-23	40,305.24	2,779.69	819.75	43,904.67
12-21	62,109.93	19.59	0.00	62,129.52

A.3 问题 4-2

9 月 23 日购电与储能见正文表17、18。各项费用按实际交付价格结算。2025.3.20 的购电和储能结果分别见表39和表40。

表 39 微网在指定时间段的购电量及全天的购电量和购电费（2025.3.20）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	605.05	14:00–14:10	0.00
16:00–16:10	493.66	18:00–18:10	711.53	20:00–20:10	271.08
全天购电量		68,314.47	全天购电费		42,627.22

表 40 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.3.20）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4,477.53	96.73	4:00–8:00	776.94	5,581.54
8:00–12:00	6,214.23	2,777.38	12:00–16:00	3,718.36	415.81
16:00–20:00	206.46	5,950.42	20:00–24:00	5,735.02	2,666.85
0:00 储电量		6,809.84	24:00 储电量		6,393.60

2025.6.21 的购电和储能结果分别见表41和表42。

表 41 微网在指定时间段的购电量及全天的购电量和购电费（2025.6.21）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	247.19	18:00–18:10	518.51	20:00–20:10	0.00
全天购电量		38,164.95	全天购电费		19,640.47

表 42 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.6.21）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	766.73	4,213.83	4:00–8:00	1,369.00	1,719.95
8:00–12:00	8,448.53	0.00	12:00–16:00	0.00	0.00
16:00–20:00	0.00	5,185.01	20:00–24:00	7,302.26	3,091.92
0:00 储电量		7,867.26	24:00 储电量		8,175.44

2025.12.21 的购电和储能结果分别见表43和表44。

表 43 微网在指定时间段的购电量及全天的购电量和购电费（2025.12.21）

时间段	购电量	时间段	购电量	时间段	购电量	
10:00–10:10	0.00	12:00–12:10	994.23	14:00–14:10	0.00	
16:00–16:10	855.14	18:00–18:10	733.64	20:00–20:10	0.00	
全天购电量		99,790.43		全天购电费		75,769.21

表 44 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.12.21）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4,580.59	57.06	4:00–8:00	809.79	1,385.35
8:00–12:00	5,931.88	6,811.36	12:00–16:00	6,190.28	2,220.11
16:00–20:00	0.00	5,577.31	20:00–24:00	5,748.19	2,842.53
0:00 储电量		6,676.37	24:00 储电量		6,618.00

表45按题面表 3 横向列出四个日期的全部紧急购电事件，表46给出含紧急费的完整账单。

表 45 微网在指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
无	0.00	无	0.00	18:20–18:30	1.08	无	0.00
				20:10–22:00	900.32		
全天合计	0.00	全天合计	0.00	全天合计	901.41	全天合计	0.00

表 46 问题 4-2 指定日期的全天账单（元）

日期	原计划费	调整费	紧急费	总费用
03-20	42,627.22	0.00	0.00	42,627.22
06-21	19,640.47	0.00	0.00	19,640.47
09-23	44,011.52	0.00	5,750.71	49,762.23
12-21	75,769.21	0.00	0.00	75,769.21

A.4 问题 4-3

9 月 23 日购电与储能见正文表19、20。原计划与最终购电分列，费用均按实际交付价格结算。2025.3.20 的购电和储能结果分别见表47和表48。

表 47 微网在指定时间段的购电量及全天的购电量和购电费（2025.3.20）

零时原计划					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	557.73	14:00–14:10	0.00
16:00–16:10	553.12	18:00–18:10	708.74	20:00–20:10	169.45
全天购电量		66,482.67	全天购电费		41,467.81
最终常规购电					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	557.73	14:00–14:10	0.00
16:00–16:10	553.12	18:00–18:10	708.74	20:00–20:10	169.45
全天购电量		67,423.97	全天购电费		42,600.22

表 48 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.3.20）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4,611.97	198.08	4:00–8:00	297.63	5,615.82
8:00–12:00	5,764.68	2,777.38	12:00–16:00	4,814.41	262.75
16:00–20:00	21.19	5,841.32	20:00–24:00	5,613.95	2,794.52
0:00 储电量		6,689.07	24:00 储电量		6,267.33

2025.6.21 的购电和储能结果分别见表49和表50。

表 49 微网在指定时间段的购电量及全天的购电量和购电费（2025.6.21）

零时原计划					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	204.64	18:00–18:10	489.64	20:00–20:10	0.00
全天购电量		36,155.25	全天购电费		18,303.04
最终常规购电					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	204.64	18:00–18:10	489.64	20:00–20:10	0.00
全天购电量		36,184.92	全天购电费		18,336.05

表 50 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.6.21）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	510.14	4,223.53	4:00–8:00	1,295.43	1,721.18
8:00–12:00	9,399.81	0.00	12:00–16:00	0.00	0.00
16:00–20:00	0.00	5,385.41	20:00–24:00	6,970.19	3,091.92
0:00 储电量		7,320.39	24:00 储电量		7,653.92

2025.12.21 的购电和储能结果分别见表51和表52。

表 51 微网在指定时间段的购电量及全天的购电量和购电费（2025.12.21）

零时原计划					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	959.50	14:00–14:10	0.00
16:00–16:10	849.15	18:00–18:10	723.11	20:00–20:10	0.00
全天购电量		96,784.31	全天购电费		72,861.20
最终常规购电					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	959.50	14:00–14:10	0.00
16:00–16:10	849.15	18:00–18:10	723.11	20:00–20:10	0.00
全天购电量		96,795.52	全天购电费		72,885.14

表 52 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.12.21）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4,626.56	131.98	4:00–8:00	1,228.44	1,461.20
8:00–12:00	5,780.01	7,205.31	12:00–16:00	6,950.41	2,283.94
16:00–20:00	10.53	5,708.00	20:00–24:00	5,611.05	2,857.70
0:00 储电量		6,386.93	24:00 储电量		6,341.96

表53按题面表 3 横向列出四个日期的全部紧急购电事件，表54给出含紧急费的完整账单。

表 53 微网在指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
无	0.00	无	0.00	18:20–18:30	1.08	无	0.00
				20:10–20:30	16.61		
				20:50–21:10	87.54		
				21:20–22:00	76.78		
全天合计	0.00	全天合计	0.00	全天合计	182.01	全天合计	0.00

表 54 问题 4-3 指定日期的全天账单（元）

日期	原计划费	调整费	紧急费	总费用
03-20	41,467.81	1,132.41	0.00	42,600.22
06-21	18,303.04	33.01	0.00	18,336.05
09-23	42,202.79	2,700.80	895.75	45,799.34
12-21	72,861.20	23.94	0.00	72,885.14

B 补充对照与参数明细

B.1 设备与预测参数检验

表55列出问题一的设备扰动结果，各组保持相同输入曲线与 6000 kWh 首末库存，每次只改变一个参数。

表 55 问题一的单参数扰动与最低电费

扰动参数	取值	费用/元	费用变化/%
可用容量/kWh	7680	36,704.73	4.49
可用容量/kWh	9600	35,126.95	0.00
可用容量/kWh	11520	33,868.53	-3.58
功率上限/kW	4000	35,247.87	0.34
功率上限/kW	5000	35,126.95	0.00
功率上限/kW	6000	35,083.29	-0.12
单向效率	0.85	36,603.41	4.20
单向效率	0.9	35,126.95	0.00
单向效率	0.95	33,767.03	-3.87

表56列出八种预报组合。各组使用相同主参数、三次调整时刻和热启动，比较读取不同新预报后的运行费用。

表 56 问题 3 新小时光伏预报的完整子集对照

新增预报时刻	纯附件 3 费用（元）	融合策略费用（元）	融合紧急电（kWh）
无	13,580,285.75	13,333,061.82	29,178.88
18	13,580,285.30	13,333,062.82	29,178.86
12	13,564,801.30	13,344,238.23	29,324.32
12、18	13,564,801.43	13,344,238.54	29,324.42
06	13,585,837.75	13,335,964.19	25,744.14
06、18	13,585,837.09	13,335,963.52	25,744.09
06、12	13,511,207.99	13,315,835.68	25,862.47
06、12、18	13,511,208.12	13,315,835.99	25,862.57

表57比较不同日末预测恢复目标，结合期初、期末库存说明费用变化。

表 57 日末预测储备目标的变化（2—12 月）

目标/kWh	总费用/元	期初储电量/kWh	期末储电量/kWh
3000	13,298,180.06	4,231.74	3,105.48
6000	13,315,835.99	7,089.87	5,987.72
9000	13,346,622.47	9,942.13	8,916.25

表58、59分别比较固定与波动电价下的光伏来源，各组使用共同热启动；表60为 1 月 15—31 日选参费用，问题 4-3 固定融合权重 0.50。

表 58 问题三的光伏预测基准（共同热启动，2—12 月）

预测来源	λ	α	总费用/元
纯历史	0	0.65	13,428,215.91
纯附件三	1	0.65	13,511,208.12
融合	0.5	0.65	13,315,835.99

表 59 问题 4-3 的光伏预测基准（共同热启动，2—12 月）

预测来源	λ	α	总费用/元
纯历史	0	0.65	14,166,961.16
纯附件三	1	0.65	14,258,363.92
融合	0.5	0.65	14,044,885.25

表 60 波动电价的 1 月分位评分（1 月 15—31 日）

α	问题 4-2 费用/元	问题 4-3 费用/元
0.5	1,029,631.23	964,022.95
0.65	985,076.81	957,308.74
0.7	974,056.08	959,947.92
0.8	966,879.67	972,861.43
0.9	991,846.47	996,125.45
0.95	1,013,304.20	1,015,980.29

B.2 逐笔收费的替代合约

若日内增购成交后不可免费撤回，需逐次记录修订。令 $q^{(0)} = g$ ，第 r 次修订为 $q^{(r)}$ ，则在适用交付段收取

$$C^{\text{trade}} = \sum_{d,t} \left[p_{d,t} g_{d,t} + 5p_{d,t} e_{d,t} + \sum_r \left(1.5p_{d,t} \left(q_{d,t}^{(r)} - q_{d,t}^{(r-1)} \right)_+ + 0.5p_{d,t} \left(q_{d,t}^{(r-1)} - q_{d,t}^{(r)} \right)_+ \right) \right]. \quad (39)$$

这里 $p_{d,t}$ 取交付段的实际价格。原购电款不返还，余电又可以不利用，因此保留此前承诺的电量更省费。替代规划采用 $q_t^{(r)} \geq q_t^{(r-1)}$ ，各次新增量分别计费。

表61比较主合约、原调度按逐笔制度重新计费、逐笔制度下重新优化三种情形。重新计费沿用原调度与库存，可直接观察收费规则的影响；重新优化则从1月选参并运行，热启动及后续安排随之改变。各方案参数和首末库存见表62。

表 61 合约解释与相应策略费用（2—12 月，元）

方案	结算与策略	总费用	较无调整方案节省
问题 3	主合约	13,315,835.99	592,339.96
问题 3	冻结主调度，逐笔重计	13,642,823.29	265,352.66
问题 3	逐笔合约重新优化	13,460,502.62	447,673.33
问题 4-3	主合约	14,044,885.25	630,959.63
问题 4-3	冻结主调度，逐笔重计	14,390,498.18	285,346.69
问题 4-3	逐笔合约重新优化	14,199,964.36	475,880.51

表 62 逐笔合约重优化的参数与库存（电量单位：kWh）

方案	λ	α	紧急购电	2 月初储电量	年末储电量
问题 3	0.25	0.65	27,362.36	7,089.87	5,987.72
问题 4-3	0.25	0.65	27,071.28	7,114.54	5,997.11

重新优化后，固定与波动电价下分别比对应无调整方案节省 447673.33 元和 475880.51 元，均低于主合约的节省额。原调度按逐笔制度重新计费时费用更高，重新优化能减轻这部分增加的支出。因此，合约规则的变化还需要相应调整购电策略。

B.3 价格感知储能反馈对照

为考察未来电价对储能控制的影响，每个十分钟段保持剩余常规购电量 q 固定，采用最近一次允许更新时得到的预测，并将第一段净负荷和价格替换为当段观测值，求解

$$\min \sum_{k=t}^{143} 5\hat{p}_k e_k, \quad c_k \leq (q_k - \tilde{N}_k)_+, \quad b_k \leq (\tilde{N}_k - q_k)_+. \tag{40}$$

模型以实际库存为初值，满足能量平衡、功率和库存边界。充电使用常规富余电，紧急购电补足缺口，午夜剩余库存价值取 0。每次求解后只执行第一段动作；到允许的发布时间，再更新完整预测并修改常规购电计划。

两种反馈采用相同参数、预报发布时间、1 月热启动及正式期初库存，后续计划根据各自实际储电量更新。表63比较常规费、紧急费与总费，表64给出增加的费用和首末库存；费用增量按滚动反馈总费减去原反馈总费计算。

表 63 确定性价格感知反馈与原反馈的费用比较（2—12 月，元）

方案	反馈规则	常规购电费	紧急购电费	总费用
问题 2	贪心	13,291,625.72	616,550.23	13,908,175.95
问题 2	确定性滚动	13,287,773.99	654,732.22	13,942,506.21
问题 3	贪心	13,177,405.10	138,430.89	13,315,835.99
问题 3	确定性滚动	13,062,906.94	466,563.01	13,529,469.95
问题 4-2	贪心	14,032,328.84	643,516.03	14,675,844.87
问题 4-2	确定性滚动	14,028,077.93	681,762.73	14,709,840.66
问题 4-3	贪心	13,917,214.62	127,670.62	14,044,885.25
问题 4-3	确定性滚动	13,807,269.57	451,105.26	14,258,374.82

表 64 滚动反馈的费用增量与物理统计

方案	增费（元）	紧急电（kWh）	未利用电（kWh）	期初 SOC（kWh）	期末 SOC（kWh）
问题 2	34,330.27	147,630.86	2,692,215.80	6,840.33	6,258.37
问题 3	213,633.96	128,007.58	2,053,973.08	7,089.87	5,987.72
问题 4-2	33,995.79	143,395.60	2,674,373.02	6,864.27	6,263.02
问题 4-3	213,489.58	120,113.54	2,040,532.00	7,114.54	5,997.11

四组滚动反馈均降低了常规费，但紧急费增加得更多，因此总支出上升。各组期末库存与对应原反馈相同，费用可在相同储备水平下比较。本次沿用原参数，跨日库存价值也未作估计，后续改进需要进一步考虑这两个因素。

B.4 上月选参的滚动评价

在 2025 年 4—12 月 275 天比较逐月选参：每月开始前，根据上一个完整月的费用，从 30 个候选中重选融合权重和分位；历史光伏基准在 $\lambda = 0$ 下选择分位。各候选从同一参考月初库存模拟，仅按实际费用评分。候选月底库存用于记录该次模拟结果，正式运行继续承接自身库存。

固定参数、逐月选参的融合预测、逐月选参的历史预测均从相同 4 月 1 日库存开始，之后各自连续运行。表65列出累计账单及首末库存。费用统计为 4—12 月，与主结果 2—12 月的区间不同，比较应采用各自对应的基准。

表 65 按月选参与固定参数比较（2025 年 4—12 月）

方案	参数规则	总费用（元）	紧急电（kWh）	期初 SOC（kWh）	期末 SOC（kWh）
问题 3	固定 1 月参数	11,025,848.81	24,392.62	6,461.68	5,987.72
问题 3	上月选择融合	11,027,957.95	39,352.86	6,461.68	5,987.72
问题 3	上月选择纯历史	11,132,033.28	37,016.85	6,461.68	5,987.72
问题 4-3	固定 1 月参数	11,695,213.43	23,070.56	6,517.67	5,997.11
问题 4-3	上月选择融合	11,711,534.09	39,935.96	6,517.67	5,997.11
问题 4-3	上月选择纯历史	11,812,014.14	46,519.39	6,517.67	5,997.11

问题三和 4-3 中，逐月选参的融合方案比固定参数分别多支出 2109.14 元和 16320.66 元，9 个月中分别只有 3 个月和 1 个月更省。与逐月选参的历史预测比较，融合方案仍分别节省 104075.33 元和 100480.05 元。在本年度的比较中，融合预测仍有费用优势，逐月选参的效果则较有限；其统计显著性和其他年度的表现还需更充分的数据检验。

C 计算程序源码

C.1 数据预处理程序

prepare_data.py

```
1  """Validate source workbooks and expose explicit, reproducible time conventions.
2
3  Main arrays use right-endpoint power as the representative 10-minute power.
4  The *_trapezoid arrays provide the alternative piecewise-linear integral.
5  No fitted predictor, future-observation imputation, or outlier deletion is used.
6  """
7
8  from __future__ import annotations
9
10 import argparse
11 from datetime import date, datetime, time, timedelta
12 import hashlib
13 import json
14 from pathlib import Path
15
16 import numpy as np
17 import openpyxl
18
19
20 ROOT = Path(__file__).resolve().parents[1]
21 EXPECTED_DATES = [date(2025, 1, 1) + timedelta(days=i) for i in range(365)]
22 RELEASE_HOURS = (0, 6, 12, 18)
23
24
25 def minutes(value: object) -> int:
26     """Convert mixed Excel time cells, preserving the explicit next-day mark."""
27     if isinstance(value, datetime):
28         value = value.time()
29     if isinstance(value, time):
30         if value.second or value.microsecond:
31             raise ValueError(f"Unexpected fractional time: {value!r}")
32         return value.hour * 60 + value.minute
33     if isinstance(value, str):
34         text = value.strip()
35         day_offset = 1440 if text.endswith("+1") else 0
36         if day_offset:
37             text = text[:-2]
38         hour, minute = map(int, text.split(":"))
39         return day_offset + hour * 60 + minute
40     raise ValueError(f"Unsupported time cell: {value!r}")
41
```

```

42
43 def calendar_date(value: object) -> date:
44     if isinstance(value, datetime):
45         return value.date()
46     if isinstance(value, date):
47         return value
48     if isinstance(value, str):
49         return date(*map(int, value.split("-")))
50     raise ValueError(f"Unsupported date: {value!r}")
51
52
53 def checked_numeric(rows: list, shape: tuple[int, ...], label: str) -> np.ndarray:
54     # float(None) fails rather than silently treating a blank as zero.
55     out = np.asarray([[float(v) for v in row] for row in rows], dtype=np.float64)
56     if out.shape != shape:
57         raise ValueError(f"{label}: expected {shape}, received {out.shape}")
58     if not np.all(np.isfinite(out)):
59         raise ValueError(f"{label}: contains non-finite values")
60     if np.any(out < 0):
61         raise ValueError(f"{label}: contains negative values; inspect source")
62     return out
63
64
65 def read_daily(sheet, label: str) -> np.ndarray:
66     rows = list(sheet.values)
67     if [minutes(v) for v in rows[0][1:]] != list(range(10, 1441, 10)):
68         raise ValueError(f"{label}: time columns must run from 00:10 through 24:00")
69     if [calendar_date(row[0]) for row in rows[1:]] != EXPECTED_DATES:
70         raise ValueError(f"{label}: dates must be every day of 2025 in order")
71     return checked_numeric([row[1:] for row in rows[1:]], (365, 144), label)
72
73
74 def trapezoid_daily(values: np.ndarray) -> np.ndarray:
75     """Interval means for a continuous annual piecewise-linear power curve.
76
77     The one missing annual origin is held equal to the first sample. Subsequent
78     days start at the immediately preceding day's 24:00 observation.
79     """
80     flat = values.ravel()
81     left = np.concatenate([flat[0]], flat[:-1])
82     return ((left + flat) / 2).reshape(values.shape)
83
84
85 def summarize(values: np.ndarray) -> dict:
86     return {
87         "shape": list(values.shape),
88         "minimum": float(np.min(values)),
89         "maximum": float(np.max(values)),

```

```

90         "mean": float(np.mean(values)),
91         "missing_or_nonfinite": int(np.count_nonzero(~np.isfinite(values))),
92         "negative_count": int(np.count_nonzero(values < 0)),
93     }
94
95
96 def prepare(raw: Path, output: Path) -> dict:
97     paths = {i: raw / f"附件{i}.xlsx" for i in range(1, 5)}
98     with paths[1].open("rb") as stream:
99         wb = openpyxl.load_workbook(stream, read_only=True, data_only=True)
100         try:
101             rows = list(wb["Sheet1"].values)
102             finally:
103                 wb.close()
104         if [minutes(row[0]) for row in rows[1:]] != list(range(10, 1441, 10)):
105             raise ValueError("Attachment 1: invalid time sequence")
106         q1 = checked_numeric([row[1:] for row in rows[1:]], (144, 3), "Attachment 1")
107
108         wb = openpyxl.load_workbook(paths[2], read_only=True, data_only=True)
109         try:
110             load = read_daily(wb["小区负载"], "Load")
111             pv = read_daily(wb["光伏发电实际功率"], "Actual PV")
112             finally:
113                 wb.close()
114         wb = openpyxl.load_workbook(paths[4], read_only=True, data_only=True)
115         try:
116             price = read_daily(wb["Sheet1"], "Dynamic price")
117             finally:
118                 wb.close()
119         wb = openpyxl.load_workbook(paths[3], read_only=True, data_only=True)
120         try:
121             forecast_rows = list(wb["Sheet1"].values)
122             finally:
123                 wb.close()
124         if list(forecast_rows[0][2:]) != [f"预报{i}小时" for i in range(1, 25)]:
125             raise ValueError("Attachment 3: forecast horizons are not hours +1 through +24")
126         if len(forecast_rows) != 1461:
127             raise ValueError("Attachment 3: expected 1460 issue rows")
128         inherited = None
129         for i, row in enumerate(forecast_rows[1:]):
130             if row[0] not in (None, ""):
131                 inherited = calendar_date(row[0])
132                 expected_date = EXPECTED_DATES[i // 4]
133                 expected_hour = RELEASE_HOURS[i % 4]
134                 if inherited != expected_date or minutes(row[1]) != expected_hour * 60:
135                     raise ValueError(f"Attachment 3: incorrect issue date/hour at row {i + 2}")
136         forecast_hourly = checked_numeric(
137             [row[2:] for row in forecast_rows[1:]], (1460, 24), "PV forecasts"

```

```

138     ).reshape(365, 4, 24)
139
140     # The interpolation origin is available at issue time, not after it.
141     anchors = np.zeros((365, 4), dtype=np.float64)
142     anchors[1:, 0] = pv[:-1, -1]
143     for issue_index, hour in enumerate(RELEASE_HOURS[1:], start=1):
144         anchors[:, issue_index] = pv[:, hour * 6 - 1]
145     hourly_knots = np.concatenate((anchors[...], None), forecast_hourly), axis=2)
146     fine_knots = np.empty((365, 4, 145), dtype=np.float64)
147     for d in range(365):
148         for issue_index in range(4):
149             fine_knots[d, issue_index] = np.interp(
150                 np.arange(145) / 6, np.arange(25), hourly_knots[d, issue_index]
151             )
152
153     arrays = {
154         "dates": np.array([d.isoformat() for d in EXPECTED_DATES]),
155         "load": load,
156         "pv": pv,
157         "fixed_price": q1[:, 0],
158         "dynamic_price": price,
159         "q1_load": q1[:, 1],
160         "q1_pv": q1[:, 2],
161         "forecast": fine_knots[:, :, 1:],
162         "load_trapezoid": trapezoid_daily(load),
163         "pv_trapezoid": trapezoid_daily(pv),
164         "q1_load_trapezoid": (np.roll(q1[:, 1], 1) + q1[:, 1]) / 2,
165         "q1_pv_trapezoid": (np.roll(q1[:, 2], 1) + q1[:, 2]) / 2,
166         "forecast_trapezoid": (fine_knots[:, :, :-1] + fine_knots[:, :, 1:]) / 2,
167         "forecast_hourly": forecast_hourly,
168         "forecast_anchor": anchors,
169         "forecast_release_hours": np.array(RELEASE_HOURS, dtype=np.int64),
170         "interval_end_minutes": np.arange(10, 1441, 10, dtype=np.int64),
171         "dt_hours": np.array(1 / 6),
172     }
173
174     # At exact hourly targets, the right-endpoint interpolant must reproduce
175     # the supplied forecasts; no day shift or averaging is allowed here.
176     np.testing.assert_allclose(arrays["forecast"][:, :, 5::6], forecast_hourly,
177                                rtol=0, atol=1e-10)
178
179     # Integration of the linear interpolant must preserve hourly trapezoids.
180     np.testing.assert_allclose(
181         arrays["forecast_trapezoid"].reshape(365, 4, 24, 6).mean(axis=3),
182         (hourly_knots[:, :, :-1] + hourly_knots[:, :, 1:]) / 2,
183         rtol=1e-13, atol=1e-10,
184     )
185
186     output.mkdir(parents=True, exist_ok=True)
187     np.savez_compressed(output / "data.npz", **arrays)
188
189     audit = {

```

```

186         "source_sha256": {p.name: hashlib.sha256(p.read_bytes()).hexdigest()
187                             for p in paths.values()}},
188         "date_start": "2025-01-01",
189         "date_end": "2025-12-31",
190         "days": 365,
191         "submission_start": "2025-02-01",
192         "submission_days": 334,
193         "interval_minutes": 10,
194         "main_power_convention": "right_endpoint_representative_interval_mean",
195         "forecast_origin": "PV observation at issue time; Jan 1 00:00 = 0 kW",
196         "statistics": {key: summarize(value) for key, value in arrays.items()
197                         if key != "dates"},
198         "annual_energy_kwh": {"load": float(load.sum() / 6),
199                               "pv": float(pv.sum() / 6),
200                               "load_trapezoid": float(arrays["load_trapezoid"].sum() / 6),
201                               "pv_trapezoid": float(arrays["pv_trapezoid"].sum() / 6)},
202         "q1_energy_kwh": {"load": float(q1[:, 1].sum() / 6),
203                             "pv": float(q1[:, 2].sum() / 6)},
204         "a1_versus_annual_same_time_average_max_abs_kw": {
205             "load": float(np.max(np.abs(load.mean(axis=0) - q1[:, 1]))),
206             "pv": float(np.max(np.abs(pv.mean(axis=0) - q1[:, 2]))),
207         },
208         "duplicate_daily_rows": {"load": 365 - len(np.unique(load, axis=0)),
209                                   "pv": 365 - len(np.unique(pv, axis=0)),
210                                   "dynamic_price": 365 - len(np.unique(price, axis=0))},
211         "versions": {"numpy": np.__version__, "openpyxl": openpyxl.__version__},
212     }
213     (output / "audit.json").write_text(json.dumps(audit, ensure_ascii=False, indent=2) + "\n",
214                                         encoding="utf-8")
215     return audit
216
217
218 def main() -> None:
219     parser = argparse.ArgumentParser(description=__doc__)
220     parser.add_argument("--raw", type=Path, default=ROOT / "data" / "raw")
221     parser.add_argument("--output", type=Path, default=ROOT / "data" / "processed")
222     args = parser.parse_args()
223     audit = prepare(args.raw, args.output)
224     print(json.dumps({"output": str(args.output / "data.npz"),
225                       "days": audit["days"],
226                       "annual_energy_kwh": audit["annual_energy_kwh"]},
227                     ensure_ascii=False, indent=2))
228
229
230 if __name__ == "__main__":
231     main()

```

C.2 公共模型与求解工具

```
model.py
1  """Causal forecasts, risk-quantile linear planning and feasible battery feedback.
2
3  All power input is kW; optimization and execution variables are AC-side kWh.
4  """
5  from __future__ import annotations
6
7  from dataclasses import dataclass
8  from functools import lru_cache
9  import numpy as np
10 from scipy.optimize import linprog
11 from scipy.sparse import lil_matrix, csr_matrix, hstack
12
13 DT = 1 / 6
14 ETA = 0.9
15 LOW, HIGH, INITIAL = 1200.0, 10800.0, 6000.0
16 LIMIT = 5000 * DT
17
18
19 @lru_cache(maxsize=8)
20 def lp_structure(n, eta=ETA):
21     # x = [grid, charge, discharge, state_after_slot, unused_energy]
22     a = lil_matrix((2 * n, 5 * n))
23     for t in range(n):
24         a[t, t], a[t, n+t], a[t, 2*n+t], a[t, 4*n+t] = 1, -1, 1, -1
25         a[n+t, n+t], a[n+t, 2*n+t], a[n+t, 3*n+t] = -eta, 1/eta, 1
26         if t:
27             a[n+t, 3*n+t-1] = -1
28     return a.tocsr()
29
30
31 def optimize(net, price, soc, base=None, refund=False, terminal=INITIAL, tie_break=True,
32             terminal_equal=False, eta=ETA, settlement='final_net'):
33     """Deterministic LP for the remaining delivery slots.
34
35     ``base`` is the midnight nomination under final-net settlement, or the last
36     committed nomination under per-revision settlement. Previously incurred
37     costs are sunk. With positive prices, free disposal and no refund, lowering
38     that base is dominated by keeping the commitment and discarding its energy.
39     """
40     if settlement not in ('final_net', 'per_revision'):
41         raise ValueError(f'Unknown settlement: {settlement}')
42     if settlement == 'per_revision' and refund:
43         raise ValueError('refund=True is only supported with final_net settlement')
44     net, price = np.asarray(net, float), np.asarray(price, float)
```

```

45     n = len(net)
46     a = lp_structure(n,eta)
47     b = np.r_[net, soc, np.zeros(n-1)]
48     cost = np.zeros(5*n)
49     cost[:n] = price if base is None else 1.5 * price
50     if tie_break:
51         cost[n:3*n] = 1e-6
52         cost[4*n:] = 1e-7
53     lower = np.zeros(5*n)
54     upper = np.full(5*n, np.inf)
55     lower[3*n:4*n], upper[3*n:4*n] = LOW, HIGH
56     upper[n:3*n] = LIMIT
57     lower[4*n-1] = terminal
58     if terminal_equal:
59         upper[4*n-1] = terminal
60     if base is not None and not refund:
61         lower[:n] = base
62     aub, bub = None, None
63     if base is not None and refund:
64         # q=base+up-down; cost=1.5*p*up-0.5*p*down.
65         m = lil_matrix((n, 7*n))
66         for t in range(n):
67             m[t,t], m[t,5*n+t], m[t,6*n+t] = 1, -1, 1
68         from scipy.sparse import vstack
69         a = vstack([hstack([a, csr_matrix((2*n,2*n))]), m.tocsr()]).tocsr()
70         b = np.r_[b, base]
71         cost[:n] = 0
72         cost = np.r_[cost, 1.5*price, -0.5*price]
73         lower = np.r_[lower, np.zeros(2*n)]
74         upper = np.r_[upper, np.full(n,np.inf), base]
75     result = linprog(cost, A_eq=a, b_eq=b, bounds=np.c_[lower,upper], method='highs')
76     if not result.success:
77         raise RuntimeError(f'LP failed: {result.message}; n={n}, soc={soc}')
78     x = result.x
79     return dict(grid=x[:n], charge=x[n:2*n], discharge=x[2*n:3*n],
80                soc=np.r_[soc,x[3*n:4*n]], spill=x[4*n:5*n],
81                objective=float(result.fun), primal_residual=float(np.max(np.abs(a@x-b))))
82
83
84 def q1_solution(data, trapezoid=False, eta=ETA):
85     """Compatibility entry point; question-one implementation lives in q1.py."""
86     from q1 import q1_solution as solve
87     return solve(data, trapezoid=trapezoid, eta=eta)
88
89
90 def weighted_profile(history, day, weekly=True):
91     """Recent same-weekday profiles. No actual from day or later is read."""
92     if day == 0:

```

```

93         # Explicit cold-start prior; only January initialization is affected.
94         return np.full(144, 4500.0) if weekly else np.zeros(144)
95     if weekly and day >= 7:
96         idx = np.arange(day-7, max(-1,day-29), -7)
97         weights = 0.55 * np.arange(len(idx))
98     else:
99         idx = np.arange(day-1, max(-1,day-8), -1)
100        weights = 0.8 * np.arange(len(idx))
101    return np.average(history[idx], axis=0, weights=weights)
102
103
104    def historical_pv(history, day):
105        if day == 0:
106            return np.zeros(144)
107        idx = np.arange(day-1, max(-1,day-8), -1)
108        return np.average(history[idx], axis=0, weights=0.75*np.arange(len(idx)))
109
110
111    def published_pv(data, day, issue, forecast_sources=None, anchor_updates=True):
112        """Available hourly forecast, aligned to absolute targets with a causal anchor.
113
114        forecast_sources[r] is the latest allowed release row at decision r.
115        The default route preserves the original preprocessed forecast exactly.
116        """
117        sources = tuple(range(4)) if forecast_sources is None else tuple(forecast_sources)
118        if len(sources) != 4 or any(not isinstance(s,(int,np.integer)) or s < 0 or s > r
119                                   for r,s in enumerate(sources)):
120            raise ValueError('forecast_sources must contain four release indices available by
each issue')
121        start = issue*36
122        source = sources[issue]
123        if source == issue and (anchor_updates or issue == 0):
124            return data['forecast'][day,issue,:144-start].copy()
125        offset = (issue-source)*6
126        horizon_hours = (144-start)//6
127        if anchor_updates:
128            anchor = data['forecast_anchor'][day,issue]
129        elif offset:
130            anchor = data['forecast_hourly'][day,source,offset-1]
131        else:
132            previous_source = sources[issue-1]
133            anchor = data['forecast_hourly'][day,previous_source,(issue-previous_source)*6-1]
134        knots = np.r_[anchor,data['forecast_hourly'][day,source,offset:offset+horizon_hours]]
135        return np.interp(np.arange(1,145-start)/6,np.arange(len(knots)),knots)
136
137
138    def point_forecast(data, day, issue, use_forecast, dynamic=False, oracle_price=False,
139                       pv_weight=None, forecast_sources=None, load_update=True, anchor_updates=

```

```

True):
140     start = issue*36
141     load = weighted_profile(data['load'], day)
142     if start and load_update:
143         # Issue-time load update uses only completed 10-minute slots.
144         recent = slice(max(0,start-18),start)
145         ratio = np.mean(data['load'][day,recent])/max(np.mean(load[recent]),1)
146         load = load*np.clip(ratio,0.75,1.25)
147     weight = float(use_forecast) if pv_weight is None else float(pv_weight)
148     if not 0 <= weight <= 1:
149         raise ValueError('pv_weight must be between zero and one')
150     if weight == 0:
151         pv = historical_pv(data['pv'],day)[start:]
152     else:
153         pv = published_pv(data,day,issue,forecast_sources,anchor_updates)
154         if weight != 1:
155             pv = weight*pv+(1-weight)*historical_pv(data['pv'],day)[start:]
156     if not dynamic:
157         price = data['fixed_price'][start:]
158     elif oracle_price:
159         price = data['dynamic_price'][day,start:]
160     elif day == 0:
161         price = data['fixed_price'][start:]
162     else:
163         price = weighted_profile(data['dynamic_price'],day)
164         if start:
165             recent = slice(max(0,start-18),start)
166             ratio = np.mean(data['dynamic_price'][day,recent])/max(np.mean(price[recent]),
,0.01)
167             price = price*np.clip(ratio,0.5,1.5)
168             price = np.maximum(price[start:],0.001)
169     return (load[start:]-pv)*DT, np.asarray(price), load[start:], pv
170
171
172 @dataclass
173 class ForecastCache:
174     net: np.ndarray
175     price: np.ndarray
176     errors: np.ndarray
177     load: np.ndarray
178     pv: np.ndarray
179
180
181 def build_cache(data, forecast=False, dynamic=False, oracle_price=False,
182                pv_weight=None, forecast_sources=None, load_update=True, anchor_updates=
True):
183     shape = (365,4,144)
184     net, price, errors, load, pv = [np.full(shape,np.nan) for _ in range(5)]

```

```

185     actual = (data['load']-data['pv'])*DT
186     for day in range(365):
187         for issue in range(4):
188             t = issue*36
189             z,p,l,v = point_forecast(data,day,issue,forecast,dynamic,oracle_price,
190                                     pv_weight,forecast_sources,load_update,anchor_updates)
191             net[day,issue,t:],price[day,issue,t:] = z,p
192             load[day,issue,t:],pv[day,issue,t:] = l,v
193             errors[day,issue,t:] = actual[day,t:]-z
194     return ForecastCache(net,price,errors,load,pv)
195
196
197 def risk_forecast(cache, day, issue, quantile):
198     start = issue*36
199     net = cache.net[day,issue,start:].copy()
200     # Stored future errors are excluded explicitly. First 7 days are warmup only.
201     lo = max(7,day-28)
202     if day > lo:
203         past_errors = cache.errors[lo:day,issue,start:]
204         # Pool 7 neighboring delivery slots to stabilize small-sample quantiles.
205         pad = np.pad(past_errors,((0,0),(3,3)),mode='edge')
206         samples = np.concatenate([pad[:,j:j+len(net)] for j in range(7)],axis=0)
207         net += np.quantile(samples,quantile,axis=0)
208     return net
209
210
211 def execute_slot(grid, net, soc):
212     surplus = grid-net
213     if surplus >= 0:
214         charge = min(surplus,LIMIT,max(0,(HIGH-soc)/ETA))
215         return charge,0.0,soc+ETA*charge,0.0,surplus-charge
216     discharge = min(-surplus,LIMIT,max(0,(soc-LOW)*ETA))
217     return 0.0,discharge,soc-discharge/ETA,-surplus-discharge,0.0
218
219
220 def settle_revisions(plan, revisions, price, settlement='final_net', refund=False):
221     """Recompute adjustment charges from a frozen path of nominations.
222
223     ``plan`` and ``price`` have shape (... , slots); price may broadcast to plan.
224     ``revisions`` has shape (... , issues, slots): layer 0 is the original plan,
225     and NaN in later layers means that delivery slot was not revised. The
226     returned revision_up/down record every actual change relative to the
227     preceding commitment, even when final_net only bills the final net change.
228     Original-plan and emergency charges are excluded from adjustment_cost.
229     """
230     if settlement not in ('final_net', 'per_revision'):
231         raise ValueError(f'Unknown settlement: {settlement}')
232     if settlement == 'per_revision' and refund:

```

```

233         raise ValueError('refund=True is only supported with final_net settlement')
234     plan, revisions, price = (np.asarray(x, float) for x in (plan, revisions, price))
235     if (revisions.ndim != plan.ndim + 1 or revisions.shape[:-2] != plan.shape[:-1]
236         or revisions.shape[-1] != plan.shape[-1] or revisions.shape[-2] < 1):
237         raise ValueError('revisions must have shape (... , issues, slots) matching plan')
238     first = revisions[..., 0, :]
239     if np.any(np.isfinite(first) & ~np.isclose(first, plan, rtol=0, atol=1e-7)):
240         raise ValueError('revisions layer 0 must match the original plan')
241     current = plan.copy()
242     revision_up, revision_down = np.zeros_like(revisions), np.zeros_like(revisions)
243     for issue in range(1, revisions.shape[-2]):
244         proposed = revisions[..., issue, :]
245         next_plan = np.where(np.isfinite(proposed), proposed, current)
246         delta = next_plan - current
247         revision_up[..., issue, :] = np.maximum(delta, 0)
248         revision_down[..., issue, :] = np.maximum(-delta, 0)
249         current = next_plan
250     if settlement == 'per_revision':
251         up, down = revision_up.sum(axis=-2), revision_down.sum(axis=-2)
252     else:
253         up, down = np.maximum(current-plan, 0), np.maximum(plan-current, 0)
254     adjustment_cost = np.sum(price*(1.5*up+(-.5 if refund else .5)*down), axis=-1)
255     return dict(final=current, revision_up=revision_up, revision_down=revision_down,
256               up=up, down=down, adjustment_cost=adjustment_cost)
257
258
259 def simulate(data, cache, quantile, issues=(), refund=False, days=365, initial=INITIAL,
260             dynamic=False, progress=False, warmup_quantile=0.8, terminal=INITIAL,
261             settlement='final_net', warmup_cache=None):
262     if settlement not in ('final_net', 'per_revision'):
263         raise ValueError(f'Unknown settlement: {settlement}')
264     if settlement == 'per_revision' and refund:
265         raise ValueError('refund=True is only supported with final_net settlement')
266     shape = (days,144)
267     plan, adjusted, charge, discharge, emergency, spill = [np.zeros(shape) for _ in range
268 (6)]
269     state = np.zeros((days,145))
270     revisions = np.full((days,4,144),np.nan)
271     revision_up, revision_down = np.zeros_like(revisions), np.zeros_like(revisions)
272     daily_costs = np.zeros((days,4)) # original plan, adjustment, emergency, total
273     soc = initial
274     for day in range(days):
275         active_cache = warmup_cache if day < 31 and warmup_cache is not None else cache
276         active_quantile = warmup_quantile if day < 31 and warmup_quantile is not None else
277 quantile
278         base_net = risk_forecast(active_cache,day,0,active_quantile)
279         sol = optimize(base_net,active_cache.price[day,0],soc,terminal=terminal)
280         plan[day] = sol['grid']

```

```

279         adjusted[day] = plan[day]
280         revisions[day,0] = plan[day]
281         state[day,0] = soc
282         for t in range(144):
283             if t in issues:
284                 issue = t//36
285                 net = risk_forecast(active_cache,day,issue,active_quantile)
286                 base = adjusted[day,t:] if settlement == 'per_revision' else plan[day,t:]
287                 sol = optimize(net,active_cache.price[day,issue,t:],soc,base=base,refund=
refund,
288                               terminal=terminal,settlement=settlement)
289                 adjusted[day,t:] = sol['grid']
290                 revisions[day,issue,t:] = sol['grid']
291                 net_actual = (data['load'][day,t]-data['pv'][day,t])*DT
292                 c,d,soc,e,w = execute_slot(adjusted[day,t],net_actual,soc)
293                 charge[day,t],discharge[day,t],emergency[day,t],spill[day,t] = c,d,e,w
294                 state[day,t+1] = soc
295                 price = data['dynamic_price'][day] if dynamic else data['fixed_price']
296                 billed = settle_revisions(plan[day],revisions[day],price,settlement=settlement,
refund=refund)
297                 revision_up[day], revision_down[day] = billed['revision_up'], billed['revision_down
']
298                 basecost = price@plan[day]
299                 adjcost = billed['adjustment_cost']
300                 emcost = 5*price@emergency[day]
301                 daily_costs[day] = [basecost,adjcost,emcost,basecost+adjcost+emcost]
302                 if progress and (day+1)%60 == 0:
303                     print(f' day {day+1}: cumulative cost {daily_costs[:day+1,3].sum():,.0f}',
flush=True)
304                 return dict(plan=plan,adjusted=adjusted,charge=charge,discharge=discharge,
305                             emergency=emergency,spill=spill,soc=state,revisions=revisions,
306                             revision_up=revision_up,revision_down=revision_down, costs=daily_costs)
307
308
309 def summarize(result, start=31, end=None):
310     sl = slice(start,end)
311     costs = result['costs'][sl].sum(0)
312     return dict(plan_kwh=float(result['plan'][sl].sum()),
313                adjusted_kwh=float(result['adjusted'][sl].sum()),
314                emergency_kwh=float(result['emergency'][sl].sum()),
315                spill_kwh=float(result['spill'][sl].sum()),
316                charge_kwh=float(result['charge'][sl].sum()),
317                discharge_kwh=float(result['discharge'][sl].sum()),
318                plan_cost=float(costs[0]),adjustment_cost=float(costs[1]),
319                emergency_cost=float(costs[2]),total_cost=float(costs[3]),
320                start_soc=float(result['soc'][start,0]),end_soc=float(result['soc'][sl
] [-1,-1]),
321                emergency_days=int(np.sum(np.any(result['emergency'][sl]>1e-7,axis=1))))

```

```

solve_common.py
1  """Shared January calibration, scenario archives and per-question CLI output."""
2  from __future__ import annotations
3
4  import argparse
5  import json
6  from pathlib import Path
7  import numpy as np
8  from model import build_cache, simulate, summarize
9
10 ROOT = Path(__file__).resolve().parents[1]
11 QUANTILES = (0.5,0.65,0.7,0.8,0.9,0.95)
12 PV_WEIGHTS = (0.0,0.25,0.5,0.75,1.0)
13
14
15 class SolveContext:
16     """One dataset and shared caches; no state is retained between runs."""
17
18     def __init__(self, data):
19         self.data = data
20         self.caches, self.selected, self.calibration, self.results, self.weights =
21         {}, {}, {}, {}, {}
22         self.memo, self.archived = {}, {}
23         self.revision = dict(schema_version=1, selection={}, scenes={}, baselines={},
24                               information_ablation={}, contracts={},
25                               validation_days=[14,31], submission_days=[31,365],
26                               candidate_family_post_review=True,
27                               warmup=dict(quantile=0.8, pv_weight=1.0, ends_before_day=31,
28                                             applies_to='scenes with warmup_mode=attachment3_fixed'))
29
30     def cache_for(self, dynamic=False, weight=1.0, sources=(0,1,2,3), load_update=True,
31                  anchor_updates=True):
32         key=(dynamic, weight, tuple(sources), load_update, anchor_updates)
33         if key not in self.memo:
34             self.memo[key]=build_cache(self.data, forecast=True, dynamic=dynamic, pv_weight=
35             weight,
36
37                                     forecast_sources=sources, load_update=load_update,
38                                     anchor_updates=anchor_updates)
39
40         return self.memo[key]
41
42     def calibrate(self, dynamic, weight_candidates, issues=(36,72,108), settlement='final_net')
43     :
44         trials=[]
45         for weight in weight_candidates:

```

```

40         for tau in QUANTILES:
41             r=simulate(self.data,self.cache_for(dynamic,weight),tau,issues=issues,days
=31,
42                     dynamic=dynamic,warmup_quantile=None,settlement=settlement)
43             score=summarize(r,start=14)
44             trials.append(dict(pv_weight=weight,quantile=tau,
45                               validation_cost=score['total_cost'],
46                               validation_emergency_kwh=score['emergency_kwh']))
47             choice=min(trials,key=lambda x:(x['validation_cost'],x['pv_weight'],x['quantile']))
48             return dict(selected=choice,candidates=trials)
49
50     def scene(self,scene_id,label,weight,tau,dynamic=False,issues=(36,72,108),
51             sources=(0,1,2,3),load_update=True,anchor_updates=True,
52             settlement='final_net',warmup_mode='self'):
53         print(f'Scene {scene_id}: w={weight}, tau={tau}, {settlement}',flush=True)
54         cache=self.cache_for(dynamic,weight,sources,load_update,anchor_updates)
55         warm=self.cache_for(dynamic,1.0) if warmup_mode=='attachment3_fixed' else None
56         r=simulate(self.data,cache,tau,issues=issues,dynamic=dynamic,settlement=settlement,
57                 warmup_cache=warm)
58         self.archived[scene_id]=r
59         self.revision['scenes'][scene_id]=dict(label=label,pv_weight=weight,quantile=tau,
60             issues=list(issues),dynamic=dynamic,settlement=settlement,
61             forecast_sources=list(sources),load_update=load_update,anchor_updates=
anchor_updates,
62             warmup_mode=warmup_mode,soc_observation='own_actual',summary=summarize(r))
63         return r
64
65
66     def plain(obj):
67         if isinstance(obj,np.ndarray):
68             return obj.tolist()
69         if isinstance(obj,np.generic):
70             return obj.item()
71         raise TypeError(type(obj).__name__)
72
73
74     def run_question(name, solve):
75         """Write standalone results separately from the full verified generation."""
76         parser = argparse.ArgumentParser(description=f"Solve {name} independently")
77         parser.add_argument('--data', type=Path, default=ROOT/'data/processed/data.npz')
78         parser.add_argument('--output', type=Path, default=ROOT/'outputs/questions'/name)
79         args = parser.parse_args()
80         with np.load(args.data, allow_pickle=False) as source:
81             data = dict(source)
82             context = SolveContext(data)
83             if name == 'q1':
84                 results = solve(data)
85                 summary = {key: {k: v for k, v in result.items() if np.isscalar(v)}}

```

```

86             for key, result in results.items()}
87     else:
88         solve(context)
89         results = context.results
90         summary = dict(parameters=context.selected, pv_weights=context.weights,
91                        calibration=context.calibration,
92                        primary={key: summarize(value) for key, value in results.items()},
93                        revision=context.revision)
94     args.output.mkdir(parents=True, exist_ok=True)
95     np.savez_compressed(args.output/'dispatch.npz', **{
96         f'{name}_{key}': value for name, result in results.items()
97         for key, value in result.items()})
98     (args.output/'summary.json').write_text(
99         json.dumps(summary, default=plain, indent=2), encoding='utf-8')
100     if context.archived:
101         np.savez_compressed(args.output/'revision-dispatch.npz', **{
102             f'{name}_{key}': value for name, result in context.archived.items()
103             for key, value in result.items()})
104     print(f'Completed {name}: {args.output}', flush=True)

```

C.3 各问求解与统一运行

q1.py

```
1 """问题一：确定性单购电与储能优化，以及积分/效率口径对照。"""
2 import numpy as np
3 from model import DT, ETA, INITIAL, optimize
4 from solve_common import run_question
5
6
7 def q1_solution(data, trapezoid=False, eta=ETA):
8     suffix = '_trapezoid' if trapezoid else ''
9     net = (data['q1_load'+suffix]-data['q1_pv'+suffix])*DT
10    sol = optimize(net, data['fixed_price'], INITIAL, terminal_equal=True, eta=eta)
11    exact = optimize(net, data['fixed_price'], INITIAL, tie_break=False, terminal_equal=
    True, eta=eta)
12    sol['cost'] = float(sol['grid']@data['fixed_price'])
13    sol['unperturbed_lower_bound'] = float(exact['grid']@data['fixed_price'])
14    sol['optimality_gap_yuan'] = sol['cost']-sol['unperturbed_lower_bound']
15    sol['net'] = net
16    return sol
17
18
19 def solve(data):
20    return dict(q1=q1_solution(data),
21               q1_trapezoid=q1_solution(data, trapezoid=True),
22               q1_roundtrip90=q1_solution(data, eta=float(np.sqrt(.9))))
23
24
25 if __name__ == '__main__':
26    run_question('q1', solve)
```

q2.py

```
1 """问题二：固定电价、日前计划，按1月数据选择风险分位。"""
2 from model import simulate
3 from solve_common import run_question
4
5
6 def solve(context, *, dynamic=False):
7     name = 'q4_2' if dynamic else 'q2'
8     data = context.data
9     print(f'January calibration: {name}', flush=True)
10    fit=context.calibrate(dynamic,(0.0,),issues=())
11    tau=fit['selected']['quantile']
12    context.selected[name],context.calibration[name],context.weights[name]=tau,fit['
    candidates'],0.0
13    context.caches[name]=context.cache_for(dynamic,0.0)
```

```

14     context.results[name]=simulate(data,context.caches[name],tau,dynamic=dynamic,progress=
    True)
15
16
17 if __name__ == '__main__':
18     run_question('q2', solve)

```

q3.py

```

1  """问题三：固定电价、日内调整，联合选择光伏权重与风险分位。"""
2  from model import summarize
3  from solve_common import PV_WEIGHTS, run_question
4
5
6  def solve(context, *, dynamic=False):
7      name = 'q4_3' if dynamic else 'q3'
8      print(f'January joint PV/quantile calibration: {name}',flush=True)
9      fit=context.calibrate(dynamic,PV_WEIGHTS)
10     context.revision['selection'][name]=fit
11     choice=fit['selected']; tau,weight=choice['quantile'],choice['pv_weight']
12     context.selected[name],context.weights[name]=tau,weight
13     context.calibration[name]=[x for x in fit['candidates'] if x['pv_weight']==weight]
14     context.caches[name]=context.cache_for(dynamic,weight)
15     context.revision['baselines'][name]={}
16     for w,label,kind in [(1.0,'原纯附件3基准','pure_attachment3'),(0.0,'历史光伏基准','
historical')]:
17         baseline_choice=min((x for x in fit['candidates'] if x['pv_weight']==w),key=lambda
x:x['validation_cost'])
18         scene_id=name+('_legacy' if w==1 else '_historical')
19         context.scene(scene_id,label,w,baseline_choice['quantile'],dynamic=dynamic)
20         context.revision['scenes'][scene_id]['validation_cost']=baseline_choice['
validation_cost']
21         context.revision['baselines'][name][kind]=scene_id
22         if w == 0:
23             common_id=name+'_historical_fixed'
24             context.scene(common_id,'历史光伏基准（共同热启动）',w,baseline_choice['quantile
'],
25
26                 dynamic=dynamic,warmup_mode='attachment3_fixed')
27             context.revision['scenes'][common_id]['validation_cost']=baseline_choice['
validation_cost']
28             context.revision['baselines'][name]['historical_common_warmup']=common_id
29             context.results[name]=context.scene(name+'_selected','修订主策略',weight,tau,dynamic=
dynamic,
30
31                 warmup_mode='attachment3_fixed')
32             print(name,'selected',choice,'summary',summarize(context.results[name]),flush=True)
33
34 if __name__ == '__main__':

```

```
34     run_question('q3', solve)
```

q4.py

```
1  """问题四：动态电价下分别求解日前计划与日内调整策略。"""
2  from q2 import solve as solve_day_ahead
3  from q3 import solve as solve_intraday
4  from solve_common import run_question
5
6
7  def solve(context):
8      solve_day_ahead(context, dynamic=True)
9      solve_intraday(context, dynamic=True)
10
11
12  if __name__ == '__main__':
13      run_question('q4', solve)
```

solve.py

```
1  """Run the four questions, January-only calibration, ablations and sensitivity."""
2  from __future__ import annotations
3  import json
4  from pathlib import Path
5  import time
6  import hashlib
7  import itertools
8  import numpy as np
9  import scipy
10 from model import (DT, build_cache, simulate, summarize, settle_revisions)
11
12 from q1 import solve as solve_q1
13 from q2 import solve as solve_q2
14 from q3 import solve as solve_q3
15 from q4 import solve as solve_q4
16 from solve_common import SolveContext, PV_WEIGHTS, plain
17
18 ROOT = Path(__file__).resolve().parents[1]
19 OUT = ROOT/'outputs'
20
21
22 def workbook_payload(data, q1, results):
23     payload = dict(schemaVersion=1, slotConvention='right-end-0000-to-2400',
24                    q1=dict(plan=q1['grid'], charge=q1['charge'], discharge=q1['discharge'],
25                           socStart=q1['soc'][0], socEnd=q1['soc'][-1]))
26     for name, result in results.items():
27         days = []
```

```

28     for d in range(31,365):
29         row = dict(date=str(data['dates'][d]),plan=result['plan'][d],
30                     planCost=result['costs'][d,0],charge=result['charge'][d],
31                     discharge=result['discharge'][d],socStart=result['soc'][d,0],
32                     socEnd=result['soc'][d,-1],emergency=result['emergency'][d],
33                     emergencyCost=result['costs'][d,2],totalCost=result['costs'][d,3])
34         if name in ('q3','q4_3'):
35             row.update(adjusted=result['adjusted'][d],
36                         adjustedCost=result['costs'][d,0]+result['costs'][d,1])
37         days.append(row)
38     payload[name] = dict(days=days)
39     return payload
40
41
42 def main():
43     started = time.perf_counter()
44     (OUT/'main').mkdir(parents=True, exist_ok=True)
45     (OUT/'experiments/revision').mkdir(parents=True, exist_ok=True)
46     data = dict(np.load(ROOT/'data/processed/data.npz'))
47     variants = solve_q1(data)
48     q1, trap, roundtrip = (variants[key] for key in ('q1', 'q1_trapezoid', 'q1_roundtrip90'
49 ))
50     context = SolveContext(data)
51     solve_q2(context)
52     solve_q3(context)
53     solve_q4(context)
54     caches, selected = context.caches, context.selected
55     calibration, results, weights = context.calibration, context.results, context.weights
56     archived, revision = context.archived, context.revision
57     cache_for, calibrate, scene = context.cache_for, context.calibrate, context.scene
58
59     summary = dict(parameters=selected,calibration=calibration,
60                     pv_weights=weights,revision_experiments='../experiments/revision/
61 experiments.json',
62                     q1={k:v for k,v in q1.items() if np.isscalar(v)},
63                     q1_grid_kwh=float(q1['grid'].sum()),
64                     q1_trapezoid_cost=trap['cost'],q1_trapezoid_grid_kwh=float(trap['grid'].
65 sum()),
66                     q1_roundtrip90_cost=roundtrip['cost'],
67                     primary={k:summarize(v) for k,v in results.items()},
68                     sensitivity={},forecast_accuracy={},versions=dict(numpy=np.__version__,
69 scipy=scipy.__version__))
70     for name,cache in caches.items():
71         err = cache.errors[31:,0]
72         summary['forecast_accuracy'][name] = dict(
73             net_mae_kw=float(np.mean(np.abs(err))/DT),
74             load_mae_kw=float(np.mean(np.abs(cache.load[31:,0]-data['load'][31:]))),
75             pv_mae_kw=float(np.mean(np.abs(cache.pv[31:,0]-data['pv'][31:])))

```

```

72         price_mae=float(np.mean(np.abs(cache.price[31:,0]-(data['dynamic_price'][31:]
if '4' in name else data['fixed_price']))))))
73     # Controlled information experiments: current observations and update permissions
74     # remain available, only receipt of each new hourly forecast is switched.
75     for group,weight,tau,warmup in [('legacy',1.0,revision['scenes']['q3_legacy']['quantile
'], 'self'),
76                                     ('selected',weights['q3'],selected['q3'],'
attachment3_fixed')]:
77         subset_ids=[]
78         for bits in itertools.product((0,1),repeat=3):
79             sources=[0]
80             for issue,enabled in enumerate(bits,1):
81                 sources.append(issue if enabled else sources[-1])
82             if all(bits):
83                 scene_id='q3_'+group
84             else:
85                 bitstr=''.join(map(str,bits)); scene_id=f'q3_{group}_info_{bitstr}'
86                 scene(scene_id,'新小时预报 '+bitstr,weight,tau,sources=tuple(sources),
warmup_mode=warmup)
87                 subset_ids.append(scene_id)
88                 revision['information_ablation'][group]=dict(hourly_subsets=subset_ids)
89
90     legacy_tau=revision['scenes']['q3_legacy']['quantile']
91     scene('q3_legacy_no_updates','仅00点计划, 无日内重规划',1.0,legacy_tau,issues=())
92     scene('q3_ladder_replan','开放重规划与实际SOC, 沿用日前负荷与PV',1.0,legacy_tau,
93           sources=(0,0,0,0),load_update=False,anchor_updates=False)
94     scene('q3_ladder_load','再增加已观测负荷修正',1.0,legacy_tau,
95           sources=(0,0,0,0),anchor_updates=False)
96     revision['information_ablation']['legacy']['ladder']=['
97         'q3_legacy_no_updates','q3_ladder_replan','q3_ladder_load','q3_legacy_info_000',
q3_legacy']
98
99     for name,dynamic,without in [('q3',False,'q2'),('q4_3',True,'q4_2')]:
100         contract={}
101         price=data['dynamic_price'] if dynamic else data['fixed_price']
102         for kind,scene_id in [('legacy',name+'_legacy'),('selected',name+'_selected')]:
103             frozen=archived[scene_id]
104             billed=settle_revisions(frozen['plan'],frozen['revisions'],price,settlement='
per_revision')
105             extra=float(billed['adjustment_cost'][31:].sum()-frozen['costs'][31:,1].sum())
106             frozen_total=summarize(frozen)['total_cost']+extra
107             contract['frozen_'+kind]=dict(scene_id=scene_id,extra_cost=extra,total_cost=
frozen_total,
108                                           withdrawn_kwh=float(billed['revision_down'][31:].sum()),
109                                           saving_vs_without_adjustment=summarize(results[without])['total_cost']-
frozen_total,
110                                           reoptimized=False)
111             print(f'Per-revision contract: January calibration {name}',flush=True)

```

```

112         fit=calibrate(dynamic,PV_WEIGHTS,settlement='per_revision')
113         choice=fit['selected']; scene_id=name+'_per_revision'
114         scene(scene_id,'逐笔合约重新优化',choice['pv_weight'],choice['quantile'],dynamic=
dynamic,
115             settlement='per_revision',warmup_mode='attachment3_fixed')
116         contract.update(selection=fit,reoptimized=scene_id)
117         revision['contracts'][name]=contract
118
119     # Same forecast/controller, varying only issued updates. Parameters remain frozen.
120     for name,issues in [('q3_midnight_only',()),('q3_update06',(36,)),
121                        ('q3_update12',(72,)),('q3_update18',(108,)),
122                        ('q3_update06_12',(36,72)),('q3_update06_18',(36,108)),
123                        ('q3_update12_18',(72,108))]:
124         print(f'Ablation {name}',flush=True)
125         r = simulate(data,caches['q3'],selected['q3'],issues=issues,warmup_cache=cache_for(
False,1.0))
126         summary['sensitivity'][name] = summarize(r)
127     for key in ('q2','q3'):
128         for tau in (0.5,0.7,0.9):
129             print(f'Risk sensitivity {key} tau={tau}',flush=True)
130             issues = () if key=='q2' else (36,72,108)
131             r = simulate(data,caches[key],tau,issues=issues,
132                          warmup_cache=cache_for(False,1.0) if key=='q3' else None)
133             summary['sensitivity'][f'{key}_tau{tau}'] = summarize(r)
134     for key in ('q3','q4_3'):
135         print(f'Refund sensitivity {key}',flush=True)
136         r = simulate(data,caches[key],selected[key],issues=(36,72,108),refund=True,dynamic=
'4' in key,
137                     warmup_cache=cache_for('4' in key,1.0))
138         summary['sensitivity'][f'{key}_refund'] = summarize(r)
139     for terminal in (3000.0,9000.0):
140         print(f'Terminal reserve sensitivity {terminal}',flush=True)
141         r = simulate(data,caches['q3'],selected['q3'],issues=(36,72,108),terminal=terminal,
142                     warmup_cache=cache_for(False,1.0))
143         summary['sensitivity'][f'q3_terminal{int(terminal)}'] = summarize(r)
144     for key,forecasts,issues in [('q4_2',False,()),('q4_3',True,(36,72,108))]:
145         print(f'Perfect-price reference {key}',flush=True)
146         cache = build_cache(data,forecast=forecasts,dynamic=True,oracle_price=True,
pv_weight=weights[key])
147         warm=build_cache(data,forecast=True,dynamic=True,oracle_price=True) if forecasts
else None
148         r = simulate(data,cache,selected[key],issues=issues,dynamic=True,warmup_cache=warm)
149         summary['sensitivity'][f'{key}_known_price'] = summarize(r)
150     # Positive-price no-storage perfect-net reference: different information, no claim of
bound.
151     actual_net = np.maximum((data['load'][31:]-data['pv'][31:])*DT,0)
152     summary['ideal_no_storage_reference'] = dict(
153         fixed_cost=float(np.sum(actual_net*data['fixed_price'])),

```

```

154         dynamic_cost=float(np.sum(actual_net*data['dynamic_price'] [31:])),
155         grid_kwh=float(actual_net.sum()))
156     summary['elapsed_seconds'] = time.perf_counter()-started
157     revision['source_sha256']={p:hashlib.sha256((ROOT/p).read_bytes()).hexdigest()
158         for p in ('scripts/model.py','scripts/solve.py','scripts/solve_common.py',
159             'scripts/q1.py','scripts/q2.py','scripts/q3.py','scripts/q4.py','data/
processed/data.npz','docs/reviews/revision-plan.md')}}
160     # Publish one coherent generation after every scenario is complete.
161     payload = workbook_payload(data,q1,results)
162     (OUT/'main/results.json').write_text(json.dumps(payload,default=plain,separators=(',',
:')),encoding='utf-8')
163     arrays = {f'q1_{k}':v for k,v in q1.items() if isinstance(v,np.ndarray)}
164     for name,result in results.items():
165         arrays.update({f'{name}_{k}':v for k,v in result.items()})
166     np.savez_compressed(OUT/'main/dispatch.npz',**arrays)
167     np.savez_compressed(OUT/'experiments/revision/dispatch.npz',**{
168         f'{name}_{k}':v for name,result in archived.items() for k,v in result.items()})
169     (OUT/'experiments/revision/experiments.json').write_text(json.dumps(revision,default=
plain,indent=2),encoding='utf-8')
170     (OUT/'main/summary.json').write_text(json.dumps(summary,default=plain,indent=2),
encoding='utf-8')
171     print(f'Completed in {summary["elapsed_seconds"]:.1f}s',flush=True)
172
173
174 if __name__ == '__main__':
175     main()

```
